An Agentic Large Language Model Assistant for Kaggle Computational Notebooks

A Chrome Extension and Python Native Host with DOM-Grounded Context and Tool-Mediated Cell Operations

By

Hamza Ali

2022-GCUF-02772

Muhammad Zubair

2022-GCUF-02760

Supervisor

Rabia Shahid

Center of Data Science

A FYP/thesis submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Data Science (BS DS)

In

Center of Data Science (CDS) ,

Government College University Faisalabad (GCUF),

Faisalabad, Pakistan.

(June 2026)

Center of Data Science

The Center of Data Science (CDS) at Government College University Faisalabad (GCUF) is a forward-looking academic and research hub committed to advancing data-driven education, innovation, and societal impact. Guided by a clear vision to produce globally competitive data scientists and knowledge leaders, the Center plays a vital role in addressing real-world challenges through analytics, artificial intelligence, and emerging digital technologies. Equipped with modern infrastructure, state-of-the-art computing facilities, and advanced research laboratories, CDS provides an enabling environment for learning, experimentation, and innovation. The Center is supported by a team of highly qualified and experienced faculty, actively engaged in teaching, research, and industry collaboration. With a strong emphasis on market-based research, CDS fosters the development of practical solutions, intelligent applications, and data-driven products that add tangible value to society, contribute to economic growth, and support evidence-based decision-making across academia, industry, and the public sector.

Approval

It is certified that the contents and form of the FYP/thesis entitled "**An Agentic Large Language Model Assistant for Kaggle Computational Notebooks**" submitted by **Hamza Ali and Muhammad Zubair** have been found satisfactory for the requirement of the degree.

Supervisor: **Rabia Shahid**

Signature: _____

Date: _____

Committee Member 1: **Dr. Adeel Zahid**

Signature: _____

Date: _____

Committee Member 2: **Khuram Shahzad**

Signature: _____

Date: _____

Head of Department: **Prof. Dr. Tanvir Ahmad**

Signature: _____

Date: _____

FYP/Thesis Acceptance Certificate

Certified that final copy of BSDS FYP thesis entitled "**An Agentic Large Language Model Assistant for Kaggle Computational Notebooks**" written by **Hamza Ali** (Registration No **2022-GCUF-02772**) and **Muhammad Zubair** (Registration No **2022-GCUF-02760**), of the Center of Data Science, has been vetted by the undersigned, found complete in all respects as per GCUF Statutes/Regulations, is free of plagiarism, errors and mistakes and is accepted as partial fulfillment for award of the Bachelor of Science in Data Science (BS DS) degree. It is further certified that necessary amendments as pointed out by FYP/Thesis Evaluation members of the scholars have also been incorporated in the said FYP/thesis.

Name of Supervisor: **Rabia Shahid**

Signature: _____

Date: _____

Member 1: **Dr. Adeel Zahid**

Signature: _____

Date: _____

Member 2: **Khuram Shahzad**

Signature: _____

Date: _____

Director: **Dr. Tanvir Ahmad**

Signature: _____

Date: _____

Dedication

This work is sincerely dedicated to our beloved parents, whose unconditional love, selfless sacrifices, and constant prayers have been the foundation of everything we have achieved. Their unwavering belief in our abilities and their encouragement through every challenge have been our greatest source of strength. This work is equally dedicated to our teachers and mentors, whose guidance, knowledge, and dedication to academic excellence have shaped our thinking and inspired us to pursue meaningful research. We are deeply grateful for the wisdom and encouragement they so generously offered throughout our studies.

Hamza Ali

Muhammad Zubair

Certificate of Originality

I hereby declare that this submission is my own work and to the best of my knowledge it contains no materials previously published or written by another person, nor material which to a substantial extent has been accepted for the award of any degree or diploma at Center of Data Science at Center of Data Science (CDS) or at any other educational institute, except where due acknowledgement has been made in the FYP. Any contribution made to the research by others, with whom I have worked at Center of Data Science (CDS) or elsewhere, is explicitly acknowledged in the FYP. I also declare that the intellectual content of this FYP is the product of my own work, except for the assistance from others in the project's design and conception or in style, presentation and linguistics which has been acknowledged.

Student Name: **Hamza Ali**

Signature: _____

Student Name: **Muhammad Zubair**

Signature: _____

Acknowledgments

All praise belongs to Almighty Allah, whose blessings made the completion of this research possible. We are profoundly grateful for the strength and clarity of thought He bestowed upon us throughout this journey. We wish to express our deepest gratitude to our supervisor, Ms. Rabia Shahid, Lecturer at the Center of Data Science, Government College University Faisalabad, for her exceptional guidance, patience, and scholarly insight throughout every phase of this project. Her constructive feedback and encouragement were invaluable to the successful completion of this work. We are also grateful to Dr. Tanvir Ahmad, Head of the Center of Data Science, Government College University Faisalabad, for his visionary leadership and for creating an environment that nurtures research and innovation. Our sincere thanks go to all faculty members of the Center of Data Science whose teaching laid the knowledge foundation for this research, and to our families for their endless patience, moral support, and encouragement throughout our studies.

Hamza Ali

Muhammad Zubair

Abstract

Computational notebooks are widely used for data science learning and experimentation, yet practical assistance inside browser-based notebook editors remains limited. Existing code assistants are typically optimized for integrated development environments and file-based repositories, while Kaggle notebooks require cell-level interaction within a dynamic web interface. This Final Year Project presents an agentic large language model assistant for Kaggle computational notebooks, implemented as a Chrome extension integrated with a Python native host. The system captures notebook context through DOM-grounded scraping, maintains live and persistent JSON snapshots, and enables tool-mediated operations such as cell insertion, editing, deletion, and execution. Three interaction modes—Ask, Code, and Agentic—enforce distinct behavioural contracts: read-only explanation, copy-paste code generation with placement guidance, and batched tool dispatch under a mandatory two-phase protocol. The host validates tool arguments, coerces session metadata, and applies deterministic ordering for index-sensitive multi-cell operations. Empirical evaluation on frozen Kaggle notebook snapshots using GLM 4.7 reports the following outcomes. Ask mode achieved 89% workflow topic coverage, zero side effects, and reliable cell-level explanation and error diagnosis. Code mode passed four scenarios with runnable Python cells and empty-cell intent deferral while maintaining zero browser writes. The developed prototype demonstrates that robust in-browser notebook assistance can be achieved without privileged platform APIs by combining constrained tool orchestration, grounded context extraction, and careful host-side execution control. The proposed approach contributes a practical blueprint for safe and effective LLM-assisted workflows in web-native notebook environments, with identified paths toward retrieval-augmented context and ReAct-style feedback in future work.

Keywords: Computational notebooks, large language models, Kaggle notebooks, Chrome extension, native messaging, DOM grounding, agentic artificial intelligence, tool calling, data

science, GLM-4.7

Contents

List of Tables

List of Figures

List of Abbreviations and Symbols

Abbreviations

API	Application Programming Interface
BSDS	Bachelor of Science in Data Science
CDS	Center of Data Science
DOM	Document Object Model
GCUF	Government College University Faisalabad
LLM	Large Language Model
RAG	Retrieval-Augmented Generation

CHAPTER 1

Introduction and Motivation

Computational notebooks have become a primary medium for data science education, experimentation, and competition-oriented analytics. Platforms such as Kaggle expose browser-based notebook editors in which analysts interleave narrative markdown, executable code, and model outputs within a single document. The resulting workflow is iterative: cells are authored, reordered, executed, and revised as understanding of the data evolves. For novice and intermediate practitioners, this cycle demands fluency in both domain reasoning and editor mechanics, yet mainstream assistance remains largely decoupled from the live notebook surface.

Recent advances in large language models (Large Language Models (LLMs)) have produced capable coding assistants that answer questions, suggest snippets, and generate multi-step solutions from natural-language prompts. However, when the task environment is a proprietary web editor rather than a local integrated development environment, assistants typically operate on pasted fragments or static file exports. They cannot reliably observe cell order, execution state, or structural changes that occur as the user edits in real time. Consequently, suggestions may reference stale indices, omit required execution steps, or remain disconnected from the notebook the analyst is actually building.

This project addresses that integration gap by developing an agentic LLM assistant tailored to Kaggle notebook `/edit` pages. The system couples a Chrome extension with a Python native messaging host. The extension scrapes the editor Document Object Model (DOM) to maintain live and persistent JSON snapshots of notebook structure and cell content. The host mediates chat in Ask, Code, and Agentic modes, invokes a GLM 4.7 model with native `tool_calls` via the Cerebras Application Programming Interface (API), and dispatches browser-mediated tools that insert, edit, delete, list, and run cells. Session context is grounded in the scraped notebook

state and coerced to the active tab URL, yielding an in-browser copilot that manipulates the notebook the user sees rather than a detached code buffer.

The remainder of this document specifies the problem the assistant is designed to solve, states the research objectives and aims, summarises original contributions, and acknowledges scope limitations. A full literature review follows in the next chapter; here we establish motivation and the precise gap this work targets.

1.1 Problem Statement and Contribution

Despite widespread adoption of LLM-powered coding tools, no widely available assistant provides deep, tool-mediated integration with the Kaggle notebook editor running inside Chrome. General-purpose chat interfaces and IDE extensions assume access to repository files or user-supplied excerpts. They do not maintain a continuously updated representation of notebook cells as rendered in the browser, nor do they execute a constrained repertoire of editor operations with deterministic ordering guarantees. On Kaggle, cell identity is positional rather than stable: inserts and deletes shift indices, so batch modifications require careful sequencing. Existing assistants rarely enforce a read-then-write discipline, cap model rounds to control latency and cost, or reconcile bulk operations on the host before dispatch to the extension.

The central problem addressed by this FYP is therefore twofold. First, *context grounding*: how to extract faithful, timely notebook structure and content from a closed web application without privileged API access, and how to supply that context to an LLM during interactive assistance. Second, *reliable agentic manipulation*: how to translate model-proposed tool calls into safe, ordered sequences of DOM-level cell operations—including multi-cell insert–edit–run workflows—while binding each action to the correct browser tab and notebook session.

Answering this problem is worthwhile because Kaggle notebooks are a high-stakes learning and competition environment for data science students, including those in the Bachelor of Science in Data Science (BSDS) programme at Center of Data Science (CDS), Government College University Faisalabad (GCUF). In-browser assistance that reads the live notebook and applies vetted tools can reduce friction in exploratory analysis, support reproducible cell-level edits, and demonstrate a reusable architecture for DOM-grounded agentic systems where platform APIs are unavailable. The approach also informs design choices for future notebook copilots that must balance model autonomy with strict execution contracts.

1.1.1 Aims

The overarching aim of this project is to design, implement, and evaluate an agentic LLM assistant that operates directly on Kaggle computational notebooks within Google Chrome. Specifically, the work aims to:

- provide conversational assistance grounded in scraped notebook state rather than user-copied code alone;
- expose a bounded tool interface through which the model can inspect and modify cells in the live editor;
- enforce predictable agentic behaviour through a mandatory two-phase query–implement protocol and host-side batch reordering; and
- demonstrate feasibility on representative Kaggle `/edit` sessions relevant to data science coursework and competition workflows.

1.1.2 Research Objectives

The following research objectives guide the design and evaluation of the system:

1. **Architecture.** Specify and implement a Chrome extension and Python native messaging host that communicate securely, route chat by mode (Ask, Code, Agentic), and associate each request with the active notebook URL and browser tab identifier.
2. **Context pipeline.** Develop a DOM-scraping pipeline that serialises notebook cells into live and persistent JSON snapshots, enabling the host to inject current structure and content into LLM prompts without manual copy-and-paste.
3. **Tool-mediated cell operations.** Implement and register browser tools—`insert_cell`, `edit_cell_by_index`, `delete_by_index`, `run_cell`, `notebook_get_cell`, and `list_cells`—with session URL and `tab_id` coercion so dispatches target the correct editor instance.
4. **Agentic execution contract.** Integrate GLM 4.7 native `tool_calls` (Cerebras) under a mandatory two-phase protocol (read-only query, then implement), a maximum of two API calls per user message, fire-and-forget batch dispatch, and host-side descending reorder for bulk delete and insert operations.

5. **Run and structure detection.** Use scrape-based monitors rather than WebSocket access to infer cell execution and notebook structure changes, supporting operational feedback within platform constraints.

6. **Empirical validation.** Exercise the assistant on Kaggle notebook editing scenarios—including multi-cell workflows—and assess whether DOM-grounded context and ordered tool batches produce coherent, session-consistent modifications.

1.1.3 Original Contributions

This FYP makes the following original contributions:

- A **split extension–host architecture** for Kaggle notebook assistance, separating in-browser DOM interaction from LLM orchestration, snapshot persistence, and tool dispatch on the native host.
- A **DOM-grounded context pipeline** that maintains live and persistent JSON notebook snapshots derived from editor scraping, supplying the model with positional cell labels and content aligned to the user's current session.
- A **bounded agentic execution model** comprising a mandatory two-phase query–implement cycle, a hard limit of two model calls per user turn, and fire-and-forget batch execution with explicit tool ordering (deletes before inserts, runs last).
- **Host-side index reconciliation** that reorders bulk delete and insert tool calls in descending index order to reduce index-shift errors during multi-cell modifications.
- **Session binding mechanisms** that coerce notebook URL and `tab_id` across tool arguments, mitigating mismatches when the model omits or approximates session metadata.
- **Scrape-based operational monitoring** for cell runs and structural change detection in the absence of a documented Kaggle editor automation API.

1.1.4 Limitations

The scope of this work is deliberately constrained. The assistant targets **Kaggle notebook `/edit` pages only**; other notebook platforms, local JupyterLab instances, and non-edit Kaggle views are out of scope. Deployment is **Chrome-specific**, relying on extension APIs and native

messaging unavailable in all browsers. Agentic mode employs **fire-and-forget dispatch** without a full host-driven verification or recovery loop: the model must complete read and write phases within two calls, and execution outcomes are surfaced through monitors rather than closed-loop replanning. **LLM API limits**—latency, rate limits, and model behaviour—affect responsiveness and tool-call reliability. **DOM scraping** introduces sensitivity to front-end layout changes and incurs observational delay compared with privileged editor hooks. Finally, **execution metadata** (e.g., rich stderr traces, variable-level runtime state) is not yet fed back comprehensively to the model; a systematic verification loop remains identified future work, as discussed in later chapters.

1.2 Background and Context

1.2.1 Computational notebooks in data science education

Computational notebooks occupy a central role in contemporary data science curricula (Pérez & Granger, 2007; Kluyver et al., 2016). At Government College University Faisalabad (GCUF), the Bachelor of Science in Data Science (BSDS) programme introduces students to exploratory data analysis, statistical modelling, and machine-learning workflows through hands-on notebook exercises. Unlike traditional software projects organised as scripts or packages, notebook assignments emphasise *narrative reproducibility*: each analytical step is visible as a cell, outputs appear inline, and markdown cells document reasoning for instructors and peers (Rule et al., 2019).

Kaggle extends this pedagogical model into a public competition and portfolio platform. Learners fork existing kernels, adapt feature-engineering pipelines, and publish notebooks that combine code, visualisation, and commentary (Quaranta et al., 2021; Wang et al., 2021). The platform's browser editor, however, does not expose a documented automation API to third-party assistants. Students therefore rely on manual cell editing, copy-paste from external chat tools, or platform-native features that lack deep language-model integration (McNutt & DeLine, 2023).

1.2.2 The assistance gap on closed web editors

Mainstream coding assistants—including GitHub Copilot and general-purpose chat interfaces—were designed for file-centric development environments (Chen et al., 2021; Ziegler et al., 2024). When a student works inside Kaggle's `/edit` view, several properties diverge from

that assumption:

- **Positional cell identity.** Cells are referenced by index; inserts and deletes shift subsequent positions.
- **Mixed modalities.** Markdown and code cells interleave; assistance must respect cell type.
- **Execution coupling.** Code meaning depends on prior kernel state not visible in a static export.
- **Session binding.** Multiple notebook tabs may be open; actions must target the correct editor instance.
- **Platform closure.** No privileged hooks exist for external agents to list, edit, or run cells programmatically.

These properties motivate a client–host architecture that scrapes the live DOM, persists JSON snapshots, and dispatches vetted tools rather than treating the notebook as a single text buffer (Chasins et al., 2018; Lewis et al., 2020).

1.2.3 Role of large language models

Recent LLMs demonstrate strong code synthesis and explanation capabilities (Brown et al., 2020; Chen et al., 2021; OpenAI, 2023). Tool-augmented variants extend this capability by invoking structured APIs during generation (Schick et al., 2023; Yao et al., 2023; Qin et al., 2024). For notebook assistance, the research question is not whether a model can write pandas or scikit-learn code in isolation—benchmarks establish that baseline—but whether a *deployable system* can ground each turn in the user's current notebook and, when appropriate, execute cell operations safely (Vaithilingam et al., 2022; Hou et al., 2024).

This FYP answers that question through a Chrome extension and Python native host that implement three deliberately separated interaction modes: Ask (explain), Code (generate for copy-paste), and Agentic (dispatch tool batches).

1.3 Project Scope and Delimitations

Table 1.1 summarises what is included and excluded from the project boundary.

Table 1.1: Project scope and explicit delimitations.

Dimension	In scope	Out of scope
Platform	Kaggle `/edit` in Chrome	JupyterLab, Colab, VS Code notebooks
Model provider	GLM 4.7 via Cerebras API	Multi-provider orchestration
Agentic control	Two-call query–implement; fire-and-forget batch	Full ReAct loop with replanning
Evaluation	Harness benchmarks on frozen snapshots	Large-scale human subjects study
Deployment	Local native host + extension	Cloud-hosted SaaS assistant

Delimitations are intentional: they keep the artefact reproducible within FYP timelines while demonstrating principles transferable to other closed notebook surfaces (McNutt & De-Line, 2023).

1.4 Stakeholders and Use Cases

The assistant serves three stakeholder groups with distinct use cases:

1.4.1 Data science students

Bachelor of Science in Data Science (BSDS) students fork public Kaggle kernels for assignments and competitions. Typical tasks include understanding an unfamiliar notebook (Ask), implementing a new modelling cell without breaking upstream dependencies (Code), and batch-adding analysis sections during project work (Agentic). Mode separation lets instructors recommend Ask-only sessions during formative weeks and controlled Agentic use during capstone projects.

1.4.2 Competition participants

Kaggle competitors iterate rapidly on feature engineering and model selection under time pressure (Quaranta et al., 2022). Code mode supports quick generation of experimental cells with placement hints; Agentic mode can plan multi-cell pipelines when the user accepts review of dispatched batches. Ask mode supports post-mortem analysis of errors in failed submissions.

1.4.3 System maintainers and researchers

Developers extending the extension–host stack need reproducible benchmarks (Appendix A) and modular prompts under `testing/host/prompts/`. Researchers studying notebook copilots can use the frozen snapshot harness to compare models without live Kaggle sessions.

Table 1.2 maps common tasks to recommended modes.

Table 1.2: Representative use cases and recommended modes.

Task	Mode	Rationale
Explain an error traceback	Ask	Zero side effects; cites cell evidence
Add XGBoost training cell	Code	Runnable block; user controls insert
Build EDA + modelling section	Agentic	Multi-cell batch after structure query
Clarify empty cell intent	Ask / Code	Deferral pattern; no unsolicited code
Review competition kernel	Ask	Workflow coverage without mutation

1.5 Thesis Organisation

The remainder of this document is organised as follows. Chapter 2 surveys prior work on notebooks, LLM-assisted coding, retrieval grounding, and agentic tool use. Chapter 3 presents the research design, system architecture, implementation methodology, and evaluation protocol. Chapter 4 reports benchmark outcomes for Ask, Code, and Agentic modes. Chapter 5 interprets findings against the literature and states threats to validity. Chapter 6 concludes with contributions and future work. Chapter 7 offers recommendations for practitioners, maintainers, and researchers. Appendix A lists benchmark prompts and tool definitions for reproducibility.

CHAPTER 2

Literature Review

Chapter 1 established that mainstream LLM assistants remain poorly integrated with proprietary, browser-hosted notebook editors such as Kaggle. The problem couples two research threads: how analysts work in computational notebooks, and how modern language models can be grounded in live application state while executing structured tool actions. This chapter surveys prior work along those threads, organised by idea rather than chronology. We first examine notebook-centric data science practice and the Kaggle ecosystem; then review LLM applications to code and data science; discuss retrieval-augmented and context-grounded generation; analyse agentic models that interleave reasoning with tool use; and address integration challenges for web-based notebook environments. The chapter closes by synthesising the research gap and positioning the present FYP relative to notebook assistants, IDE copilots, and platform-native tooling.

2.1 Computational Notebooks and Data Science Workflows

Computational notebooks interleave executable code, narrative text, and rich outputs in a single literate document. Pérez and Granger (2007) introduced `IPython` as an interactive shell and architecture for scientific computing in Python, emphasising a read–eval–print loop that keeps code, results, and visualisations co-located. That design philosophy matured into the Jupyter ecosystem described by Kluyver et al. (2016), who characterised Jupyter Notebooks as a publishing format for *reproducible computational workflows*. Cells can be reordered and re-executed non-linearly; markdown supports exposition; and kernel outputs embed directly beneath the code that produced them. For data science education and practice—including

programmes such as the Bachelor of Science in Data Science (BSDS) at Government College University Faisalabad (GCUF)—notebooks lower the barrier between exploratory analysis and communicable artefact.

Empirical studies clarify how practitioners actually use this medium. Rule et al. (2018) distinguish *exploration* notebooks, in which analysts iteratively probe data, from *explanation* notebooks intended for presentation and reuse. The same structural affordances that support experimentation—mutable cell order, inline outputs, and narrative/code interleaving—also complicate maintenance when notebooks evolve from private sketches into shared deliverables. Rule et al. (2019) codify practical guidance for writing and sharing analyses in Jupyter, stressing clear sectioning, reproducible execution order, and documentation habits that later readers can follow without re-running every cell blindly.

Kaggle has become a prominent venue for notebook-centric data science at scale. Quaranta et al. (2021) released KGTorrent, a corpus of Python Jupyter notebooks harvested from Kaggle, enabling mining of structural and content patterns across thousands of public analyses. Wang et al. (2021) studied documentation practices in Kaggle notebooks through interviews and content analysis, finding that effective notebooks balance code, markdown explanation, and visual evidence—yet many notebooks remain under-documented or difficult to reuse. Quaranta et al. (2022) extended this line by eliciting collaborative best practices for computational notebooks and assessing their adoption in Kaggle-derived notebooks; experts recognise recommendations such as modular cell structure and explicit data provenance, but tool support and competition deadlines often impede consistent adherence.

Together, this literature establishes three properties relevant to the present project. First, notebook work is *cell-oriented*: meaningful units are indexed positions containing code or markdown, not abstract symbols in a flat source file. Second, workflows are *iterative and non-linear*; assistance must tolerate reordering, re-execution, and partial drafts. Third, Kaggle notebooks sit within a *web platform* whose social and competitive context shapes authoring norms. Competition deadlines, public leaderboards, and fork-and-remix culture encourage rapid experimentation over polished software engineering discipline—a tension Quaranta et al. (2022) document when recommended practices collide with time pressure.

From a systems perspective, notebooks also decouple *authoring state* from *execution state*. A cell may contain code that has not been run, code whose output is stale, or markdown that documents results from an earlier kernel session. Assistants that treat notebook files as static text

risk proposing edits that contradict execution order or ignore hidden failures in prior cells. Rule et al. (2018) note that analysts frequently pivot between exploratory and explanatory stances within the same document, interleaving throwaway diagnostics with narrative intended for peers. An effective copilot must therefore present the model with both structural layout (which cells exist, in what order, and of what type) and, where observable, signals about whether cells have been executed—even if full runtime introspection is unavailable on Kaggle. Any assistant targeting Kaggle must respect positional cell identity, mixed markdown/code structure, and the analyst's live editing session rather than a static export alone.

2.2 Large Language Models for Code and Data Science

The emergence of large language models trained on vast code corpora has reshaped software development assistance. Hou et al. (2024) provide a systematic literature review spanning model architectures, training data, and downstream software engineering tasks—including code generation, repair, summarisation, and test synthesis—concluding that LLMs are increasingly embedded across the development lifecycle but that task-specific evaluation and human factors remain active research areas. Among early demonstrations of code-capable models, Finnie-Ansley et al. (2022) examined OpenAI Codex in introductory programming education, reporting that students could solve standard exercises with model-generated solutions while raising concerns about over-reliance, assessment integrity, and the gap between generated snippets and pedagogically scaffolded understanding.

Commercial code-completion systems have since moved from research prototypes to everyday IDE features. Ziegler et al. (2024) measured GitHub Copilot's impact on developer productivity in a controlled trial, finding that access to AI suggestions reduced task completion time for a representative sample of implementation tasks. The study underscores that *context*—the code already visible in the editor—is central to suggestion quality. Copilot and analogous tools, however, assume an IDE buffer with stable file semantics. They do not natively observe notebook cell indices, markdown neighbours, or execution state on a remote platform.

Human–computer interaction research complements these performance studies by examining how programmers *experience* AI-generated code. Vaithilingam et al. (2022) compared user expectations with actual behaviour when using LLM-powered code generation tools, highlighting mismatches between anticipated automation and the verification burden users still carry. Mozannar et al. (2024) modelled user behaviour and cognitive costs in AI-assisted program-

ming, identifying phases of suggestion review, deferral, and acceptance that influence whether AI accelerates or disrupts flow. Both studies imply that trustworthy assistance requires not only fluent generation but also alignment with the user's current artefact and predictable edit semantics.

Computational notebooks introduce a distinct gap within this landscape. McNutt and DeLine (2023) conducted design probes for AI-powered code assistants *in notebooks*, arguing that notebooks afford interleaved narrative and code, rich outputs, and non-linear execution—properties that generic copilots under-exploit. Participants in their studies valued suggestions that respect cell boundaries, appear adjacent to the code under edit, and preserve the literate narrative surrounding analytics steps. Their prototypes explore inline suggestions and notebook-aware interaction models, yet remain oriented toward JupyterLab-style environments with extension APIs rather than closed, browser-only editors such as Kaggle's `/edit` interface.

The systematic review by Hou et al. (2024) further shows that most LLM4SE deployments target repository mining, bug repair, or test generation in traditional software projects. Notebook-centric tasks—markdown generation, cell reordering, competition-specific feature engineering—appear less frequently and are rarely evaluated on live web editors. Pedagogical studies such as Finnie-Ansley et al. (2022) focus on whether novices can complete exercises with generated code, not whether an agent can maintain a multi-cell analytical storyline under continuous user edits. Usability findings from Vaithilingam et al. (2022) and Mozannar et al. (2024) imply that even strong generators frustrate users when suggestions are hard to verify against the artefact in view. Pasting notebook fragments into a separate chat window exacerbates this verification gap because the assistant's context may omit neighbouring markdown, prior outputs, or cells the user deleted seconds earlier.

No surveyed system provides the combination of live DOM-grounded notebook state, native tool calling for cell insert/edit/delete/run, and session binding to a specific browser tab on Kaggle. General chat interfaces remain the de facto workaround for data science students, including those in coursework aligned with Bachelor of Science in Data Science (BSDS) learning outcomes, yet they offload integration labour onto the user. The notebook assistance literature thus motivates platform-specific integration work beyond porting IDE copilots.

2.3 Retrieval-Augmented and Context-Grounded Generation

Pure parametric knowledge inside an LLM is insufficient when answers must reflect private, volatile, or fine-grained state—such as the cells currently visible in a user's notebook. Lewis et al. (2020) introduced Retrieval-Augmented Generation (Retrieval-Augmented Generation (RAG)), conditioning generation on passages retrieved from an external corpus so that outputs cite up-to-date evidence rather than relying solely on weights frozen at training time. RAG decouples *what the model knows parametrically* from *what it is allowed to assert in this turn*, a separation that maps naturally onto assistant design: the notebook snapshot acts as a retrieved document bundle injected into the prompt.

For coding assistants, context grounding takes several forms. IDE copilots ground on open files and cursor position; repository-level systems ground on retrieved functions or documentation from version control. In closed web applications, none of these channels exist unless the client explicitly serialises application state. The Kaggle editor renders cells in the DOM; without scraping or privileged API access, an external chat panel sees only what the user copies. Static exports (downloaded `.ipynb` files) lag behind live edits and omit execution ordering cues. Consequently, *DOM-grounded context*—maintaining live and persistent JSON snapshots of cell type, index, and content—is not merely an implementation convenience but a response to the retrieval problem posed by closed platforms.

In-context learning provides a complementary mechanism: models conditioned on exemplars or structured state in the prompt can adapt behaviour without weight updates. Notebook assistance exploits this by serialising cell lists with positional labels (`Cell 0`, `Cell 1`, ...) so that tool arguments referencing indices align with user-visible structure. Unlike corpus RAG, where retrievers rank passages from a static index, notebook grounding resembles *continuous retrieval* from a single mutable document whose authoritative rendering lives in the browser rather than on disk.

The present architecture maintains two snapshot classes—*live* captures refreshed on scrape events and *persistent* copies keyed by notebook URL—so the host can inject context even when the user briefly navigates away or when a turn spans multiple model calls. This design echoes Lewis et al. (2020)'s separation of parametric memory and retrieved evidence, but the retriever is deterministic: the extension serialises whatever the editor currently displays. The limitation is context window size and staleness: if scraping is delayed or the user edits during generation, retrieved notebook state may drift. RAG literature emphasises freshness and provenance; note-

book copilots must likewise refresh snapshots before mutating operations and bind each request to a canonical session URL and tab identifier.

Closed web applications amplify grounding difficulty. Local Jupyter users can point assistants at on-disk `.ipynb` JSON; Kaggle's editor abstracts that file behind authenticated endpoints and a React DOM. Without platform cooperation, the assistant cannot subscribe to kernel messages or file-save events. DOM grounding is therefore not an inferior substitute for an API so much as the feasible channel for third-party integrators. These requirements extend RAG from document collections to *application-state corpora* maintained by a browser extension.

2.4 Agentic LLMs and Tool Use

Recent models go beyond single-shot completion by invoking external tools through structured interfaces. Schick et al. (2023) showed that language models can learn—via self-supervised demonstration—when to call APIs such as calculators or search engines and how to incorporate results into continued generation. Yao et al. (2023) proposed ReAct, interleaving chain-of-thought reasoning with environment actions in a single trajectory, improving synergy between planning and execution on knowledge-intensive and decision-making benchmarks. Qin et al. (2024) scaled tool learning to thousands of real-world APIs, highlighting both the promise and the brittleness of open-ended tool selection without domain-specific contracts.

Agentic patterns matter for notebook editing because user requests are seldom atomic. A prompt such as "add preprocessing, train a model, and evaluate" implies read–write–execute sequences across multiple cells. Unconstrained agents may hallucinate cell contents, operate on stale indices, or issue parallel edits that race with the editor's positional semantics. Production-oriented designs therefore impose *execution contracts*: bounded tool sets, phased protocols, and deterministic ordering. The present FYP adopts a mandatory two-phase *query–implement* cycle—first read-only inspection (`list_cells`, `notebook_get_cell`), then mutating operations—mirroring ReAct's alternation of reasoning and action while constraining write phases to at most two model calls per user message. Host-side reordering of bulk deletes and inserts in descending index order further mitigates index-shift errors endemic to positional notebook models.

Tool mediation also separates *planning* from *side effects*. The Python native host registers tools with the Cerebras API; the Chrome extension executes DOM operations. This split

echoes Qin et al. (2024)'s separation between language reasoning and API execution, but narrows the API surface to vetted notebook operations (`insert_cell`, `edit_cell_by_index`, `delete_by_index`, `run_cell`, `list_cells`, `notebook_get_cell`) rather than arbitrary web APIs. Schick et al. (2023) demonstrate that models can learn *when* to invoke tools; Yao et al. (2023) show that interleaving deliberation with actions improves multi-step tasks. The FYP combines these insights with domain-specific guardrails: Agentic mode is not an open-ended ReAct loop but a capped two-call protocol that forces inspection before mutation.

Fire-and-forget batch dispatch—where the host emits an ordered tool batch without waiting for per-cell acknowledgement before the model's second call—reflects latency constraints identified in interactive programming studies (Mozannar et al., 2024). Full closed-loop replanning after every cell run would exceed practical API budgets for classroom and competition settings. Instead, scrape-based monitors surface execution and structural changes asynchronously, allowing the user or a subsequent turn to react. Such narrowing trades generality for safety and testability—a deliberate response to closed-platform constraints where unvetted automation could corrupt user work or violate terms of service.

2.5 Web-Based Notebook Environments and Integration Challenges

Kaggle notebooks execute in the browser without exposing a documented automation API to third-party assistants. Unlike local JupyterLab, where extensions can hook kernel messages and editor events, integrators on Kaggle must treat the rendered editor as the source of truth. Web scraping research offers conceptual tools for this setting. Chasins et al. (2018) present Rousillon, a system for scraping hierarchical web data through programmatic interaction with page structure; although aimed at data extraction rather than authoring, it illustrates the fragility and expressiveness of DOM-level automation when no server cooperation exists.

Browser extensions occupy a pragmatic middle ground. They run with the user's consent in the same origin as the editor, can observe DOM mutations, and communicate with native hosts via Chrome's messaging channel. This architecture avoids credential sharing with remote scrapers while enabling periodic serialisation of notebook state. Challenges remain substantial:

- **Structural volatility.** Front-end refactors can break selectors; scrape-based monitors must be maintained.
- **Positional identity.** Cells are identified by index; inserts and deletes shift subsequent

indices, requiring ordered batch operations.

- **Execution observability.** Without kernel WebSocket access, run completion must be inferred from DOM changes rather than explicit events.
- **Session multiplicity.** Users may open several notebook tabs; assistance must coerce `tab_id` and notebook URL on every tool dispatch.
- **Latency and cost.** Agentic loops multiply LLM calls; hard caps and fire-and-forget batching bound responsiveness.

McNutt and DeLine (2023) identify notebook-specific affordances—inline outputs, narrative context, cell boundaries—that generic assistants ignore; the integration literature adds that *access mechanism* determines whether those affordances are machine-readable. Privileged platform APIs, local kernel bridges, and DOM scraping form a spectrum of fidelity and maintenance burden. Chasins et al. (2018) remind implementers that hierarchical web data extraction requires robust selectors and tolerance for layout variation—lessons directly applicable when Kaggle updates editor markup.

Native messaging between the extension and a Python host further mirrors how production assistants separate *trust boundaries*: the browser handles same-origin DOM access; the host stores snapshots, calls the remote LLM, and normalises tool arguments. This mirrors the split in enterprise copilots where IDE plugins collect context and cloud services run models, except here the "IDE" is a scraped web surface. Kaggle's closed editor pushes this project toward extension-mediated scraping plus native-host orchestration, accepting observational delay and layout sensitivity in exchange for user-side deployability without platform partnership.

2.6 Research Gap and Positioning

Synthesising the preceding sections reveals a convergent gap aligned with Chapter 1. Computational notebook research documents how analysts explore, explain, and collaborate on Kaggle (Rule et al., 2018; Wang et al., 2021; Quaranta et al., 2022), yet stops short of agentic systems that *mutate* live browser sessions. LLM for software engineering surveys catalog code intelligence advances (Hou et al., 2024) and Copilot-style productivity gains (Ziegler et al., 2024), but IDE-centric tools do not address positional cell workflows or markdown/code interleaving on proprietary web editors. RAG and tool-augmented models (Lewis et al., 2020; Schick et

al., 2023; Yao et al., 2023; Qin et al., 2024) supply architectural ingredients—retrieved context, API calls, interleaved reasoning—without specifying how to harvest notebook state from a closed DOM or enforce safe edit ordering.

Table 2.1 contrasts representative approaches with the present FYP along dimensions salient to Kaggle notebook assistance.

Table 2.1: Positioning of the proposed system relative to prior assistant paradigms.

Dimension	**IDE Copilot** (Ziegler et al., 2024)	**Notebook design probes** (McNutt & DeLine, 2023)	**Kaggle-native tooling**	**This FYP**
Deployment surface	Local IDE	Jupyter-oriented	Platform UI	Chrome extension + native host
Notebook context	Open files	Prototype hooks	Internal only	DOM-scraped JSON snapshots
Cell operations	Text edit	Suggested inserts	Manual UI	Tool-mediated insert/edit/delete/run
Agentic control	Single-shot completion	Exploratory	N/A	Two-phase query–implement, bounded calls
Session binding	Workspace	N/A	Account session	URL `+ tab_id` coercion

The proposed system occupies an under-explored niche: *agentic, tool-mediated notebook editing on Kaggle* `/edit` *pages* with explicit context grounding and execution contracts. It extends McNutt and DeLine (2023) by implementing a deployable split architecture rather than laboratory probes alone; it extends Copilot-style assistants by targeting positional notebook structure rather than flat source buffers; and it extends generic ReAct/tool-learning frameworks by specialising tools, enforcing read-then-write phases, and reconciling index shifts on the host before browser dispatch.

Limitations acknowledged in Chapter 1 remain grounded in this literature. Without privileged APIs, scraping cannot match kernel-level introspection; without closed-loop verification, agentic reliability depends on scrape monitors and bounded rounds rather than full replanning

(Vaithilingam et al., 2022; Mozannar et al., 2024). Future work might combine DOM grounding with richer execution feedback or platform partnerships; for the scope of this FYP, the literature justifies DOM-grounded snapshots and constrained tool batches as a principled response to closed web notebook environments.

The next chapter details the methodology used to realise this positioning—architecture, scraping pipeline, tool registry, and evaluation scenarios—building on the foundations established here.

2.7 Evaluation Methodologies for LLM Assistants

Empirical assessment of coding assistants spans productivity trials, usability studies, and task-specific benchmarks (Chen et al., 2021; Vaithilingam et al., 2022; Ziegler et al., 2024). Productivity studies measure task completion time when suggestions are available (Ziegler et al., 2024); usability studies capture expectation mismatches and verification effort (Vaithilingam et al., 2022; Mozannar et al., 2024). For tool-augmented agents, additional metrics include tool-call accuracy, API selection correctness, and hallucination of successful execution (Li et al., 2023; Patil et al., 2023; Qin et al., 2024).

Notebook assistants introduce evaluation dimensions absent from flat-file coding tasks:

- **Positional fidelity.** Do tool arguments reference valid cell indices after structural changes?
- **Mode-contract compliance.** Does Ask mode avoid writes? Does Code mode defer empty-cell code?
- **Context grounding.** Do explanations cite cells and variables present in the snapshot?
- **Dispatch correctness.** In bounded agentic settings, does the model emit read-then-write batches?

Hou et al. (2024) emphasise aligning metrics with deployment contracts rather than importing generic code-generation scores. This FYP adopts that principle: Agentic mode is scored on *tool dispatch* under a two-call cap; Ask and Code modes are scored on side-effect-free behaviour plus automated rubrics for coverage and code alignment. Human expert grading remains recommended future work (Chen et al., 2021; Wang et al., 2021).

2.8 Ethical and Pedagogical Considerations

LLM assistance in educational settings raises integrity and dependency concerns (Finnie-Ansley et al., 2022). Ask mode's read-only contract supports formative learning—students can request explanations without automated submission of solutions. Code mode requires manual insertion, preserving a deliberate human step before code enters the notebook. Agentic mode automates edits and should be positioned as an advanced workflow tool rather than a default for graded assignments without instructor policy.

From a safety perspective, constraining tools to six vetted cell operations reduces the attack surface compared with arbitrary code execution or unrestricted web API access (Patil et al., 2023; Schick et al., 2023). Session binding via URL and `tab_id` limits cross-notebook accidents when multiple kernels are open.

2.9 Synthesis: Design Principles for Notebook Agents

Drawing on the surveyed literature, five design principles informed the methodology in Chapter 3:

1. **Ground before generate.** Retrieval-augmented and in-context grounding (Lewis et al., 2020) require fresh notebook serialisation before mutating operations—implemented as mandatory round 0 reads in Agentic mode.

2. **Bound autonomy.** Agentic systems without caps risk runaway tool loops (Yao et al., 2023; Qin et al., 2024); the two-call contract and six-tool surface enforce predictability.

3. **Respect positional semantics.** Notebook cells are not files; bulk edits need deterministic reordering (Rule et al., 2019).

4. **Separate explanation from execution.** Usability research shows users need predictable edit semantics (Vaithilingam et al., 2022; Mozannar et al., 2024)—motivating Ask/Code/Agentic mode separation.

5. **Evaluate against contracts.** Metrics must match what the system actually does (Hou et al., 2024)—dispatch scoring for Agentic, side-effect-free rubrics for Ask and Code.

These principles bridge the gap between general-purpose LLM tooling and the constraints of closed Kaggle notebook editors identified throughout this chapter.

2.10 Chronological Landscape

Table 2.2 situates key prior work on a simplified timeline, illustrating how notebook practice, code LLMs, and agentic tool use converged to motivate this FYP.

Table 2.2: Simplified chronology of related research themes.

Period	Theme	Representative work
2007–2016	Notebook ecosystems	IPython (Pérez & Granger, 2007); Jupyter (Kluyver et al., 2016)
2018–2019	Notebook practice studies	Exploration vs. explanation (Rule et al., 2018); ten rules (Rule et al., 2019)
2020–2021	Scale and documentation	RAG (Lewis et al., 2020); KGTorrent (Quaranta et al., 2021); Codex (Chen et al., 2021)
2022–2023	Copilots and notebook UX	Expectation studies (Vaithilingam et al., 2022); ReAct (Yao et al., 2023); notebook assistants (McNutt & DeLine, 2023)
2023–2024	Tool learning at scale	Toolformer (Schick et al., 2023); ToolLLM (Qin et al., 2024); Copilot trial (Ziegler et al., 2024)
2024–2026	LLM4SE synthesis	Systematic reviews (Hou et al., 2024); present FYP artefact

The timeline shows that while individual ingredients—notebooks, retrieval, tools, copilots—matured over two decades, integrated agentic assistance for closed web notebook editors remains under-served. This FYP contributes at the convergence point in the final row.

CHAPTER 3

Methodology

Chapter 2 established that effective Kaggle notebook assistance requires DOM-grounded context, a bounded agentic execution contract, and a split trust boundary between browser and host. This chapter describes how those requirements were realised: research design, system architecture, component implementation, and the empirical evaluation protocol used in Chapters 4 and 5.

3.1 Research Design and Development Approach

This project follows a *design-and-development* research design centred on an engineered artefact: an agentic LLM assistant that operates on live Kaggle notebook `/edit` pages. The research question is not whether a language model can generate data-science code in isolation—that capability is well documented in prior work (Hou et al., 2024; Ziegler et al., 2024)—but whether a deployable client–host system can *ground* model reasoning in scraped notebook state and *execute* vetted cell operations reliably within platform constraints identified in Chapters 1 and 2. Success is therefore judged by whether the implemented architecture satisfies the objectives in Section 1.1.2: context pipeline fidelity, tool-mediated manipulation, session binding, and bounded agentic behaviour.

Development proceeded as an *iterative prototype cycle* rather than a linear waterfall. Each iteration targeted one integration risk: (i) serialising notebook cells from the editor DOM; (ii) routing messages between extension and host; (iii) injecting snapshots into LLM prompts across Ask, Code, and Agentic modes; (iv) registering and dispatching browser tools; and (v) enforcing the mandatory two-phase query–implement protocol with host-side batch reordering. Unit tests and monitor scripts under `testing/host/tests/` and `testing/host/scripts/` supported

regression checks during iteration (Hou et al., 2024). Empirical benchmarks are reported in Chapter 4.

The methodology deliberately separates *what the user sees* (the Kaggle editor in Chrome) from *what the model reasons over* (JSON snapshots and tool results on the host). This mirrors the trust-boundary pattern discussed in Section 2.5: the extension holds privileged DOM access within the user's browser session; the host holds API credentials, chat history, and deterministic tool orchestration. No Kaggle platform partnership or kernel WebSocket access is assumed; observability of cell runs and structural changes relies on scrape-based monitors, consistent with the limitations acknowledged in Section 1.1.4.

3.2 System Architecture Overview

The system comprises five cooperating layers, shown conceptually in Figure 3.1. User interaction begins on a Kaggle notebook `/edit` tab. A Chrome extension injects content scripts, scrapes cell structure and content, and exposes a chat user interface. Messages traverse a native messaging bridge to a locally installed Python process (`host.py`). The host loads or refreshes notebook JSON snapshots, assembles mode-specific prompts, calls GLM 4.7 through the Cerebras API, and dispatches tool commands back to the extension for DOM-level execution. Snapshot files in `live/` and `persistent/` directories provide the canonical notebook representation consumed by prompts and local read tools.

Table 3.1 summarises each layer's responsibility and primary artefacts.

Table 3.1: Major system components and their roles.

Component	Role	Key artefacts
Chrome extension	DOM scrape, chat interface, execute cell tools	`manifest.json`, `background.js`, content scripts
Native messaging bridge	Framed JSON over stdio between extension and host	Chrome native host manifest, `connectNative` port
Python host	LLM orchestration, snapshots, tool dispatch, session keys	`host.py`, `tool_registry.py`, `agentic_batch_executor.py`
Cerebras / GLM 4.7	Remote inference with native `tool_calls`	`cerebras_client.py`, `llm_provider.py`
Notebook JSON stores	Canonical cell lists for prompts and local tools	`live/`, `persistent/` under `data/notebooks/`

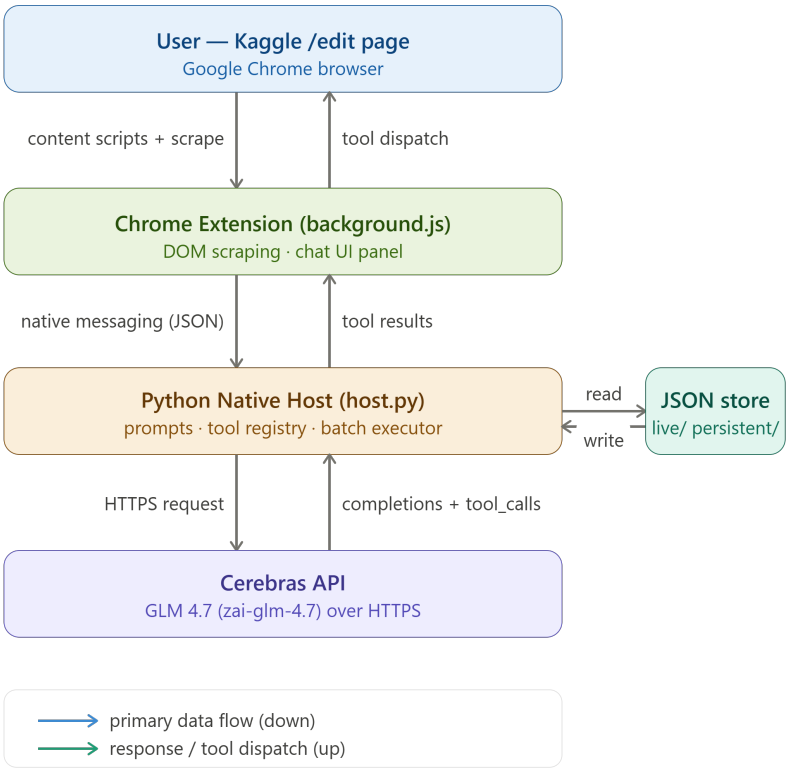

fig. f-architecture · High-level architecture of the Kaggle notebook assistant
Solid arrows = primary data flow for a single user turn in Agentic mode

Figure 3.1: High-level architecture of the Kaggle notebook assistant. Solid arrows indicate primary data flow for a single user turn in Agentic mode.

Three interaction modes govern host behaviour on each user message. **Ask** mode supplies conversational answers grounded in the current notebook snapshot without mutating cells. **Code** mode emphasises code-oriented assistance with the same grounding. **Agentic** mode enables the registered browser and local tools under a mandatory two-phase contract: round 0 is query-only (`notebook_list_cells`, `notebook_get_cell`); round 1 performs writes and runs (`insert_cell`, `edit_cell_by_index`, `delete_by_index`, `run_cell`). Agentic mode is gated by both a host environment flag (`LLM_AGENTIC_ENABLED`) and the user's mode selection in the extension interface.

3.3 Chrome Extension

The extension targets Manifest V3 and declares permissions for `tabs`, `nativeMessaging`, `scripting`, and `webNavigation`, with host permissions for `kaggle.com` and related Jupyter-proxy domains used by Kaggle's in-browser kernel. A service worker (`background.js`) maintains the native messaging port, coordinates tab-scoped scrape schedules, and routes host-issued commands to the correct frame and `tab_id`. Figure 3.2 shows the extension architecture and component structure.

Content scripts run at `document_start` on Kaggle notebook pages. They include utilities for cell indexing, kernel state observation, optional debug chat, and interface injection. Because Kaggle embeds notebook cells inside nested frames and shadow DOM trees, scraping cannot assume a flat document: the function `scrapeNotebook()` walks the document, descends into open shadow roots and accessible iframes, and collects elements matching Jupyter-style selectors (`.jp-Cell`, `.code_cell`, `.markdown_cell`, and related markers).

For each collected cell, the scraper extracts:

- positional index (application-visible, one-based where required for tool alignment);
- cell type (code versus markdown);
- source text from input areas;
- optional output text from output wrappers; and
- execution metadata where present in the DOM (prompt numbers, button titles signalling queued, running, or executed state).

Scrape events are debounced and deduplicated to limit load on both the editor and the host. Figure 3.3 illustrates the DOM scrape pipeline from editor cells to host ingestion.

When a scrape completes, the extension forwards a structured payload—notebook URL, optional Kaggle kernel identifier, `tab_id`, and the serialised cell list—to the host via native messaging. The extension also executes inverse operations when the host dispatches tool commands: inserting cells, editing source by index, deleting by index, running cells, and selecting or clicking cells to focus the editor. Each dispatch is correlated with the originating `tab_id` so that multi-tab sessions do not cross-contaminate.

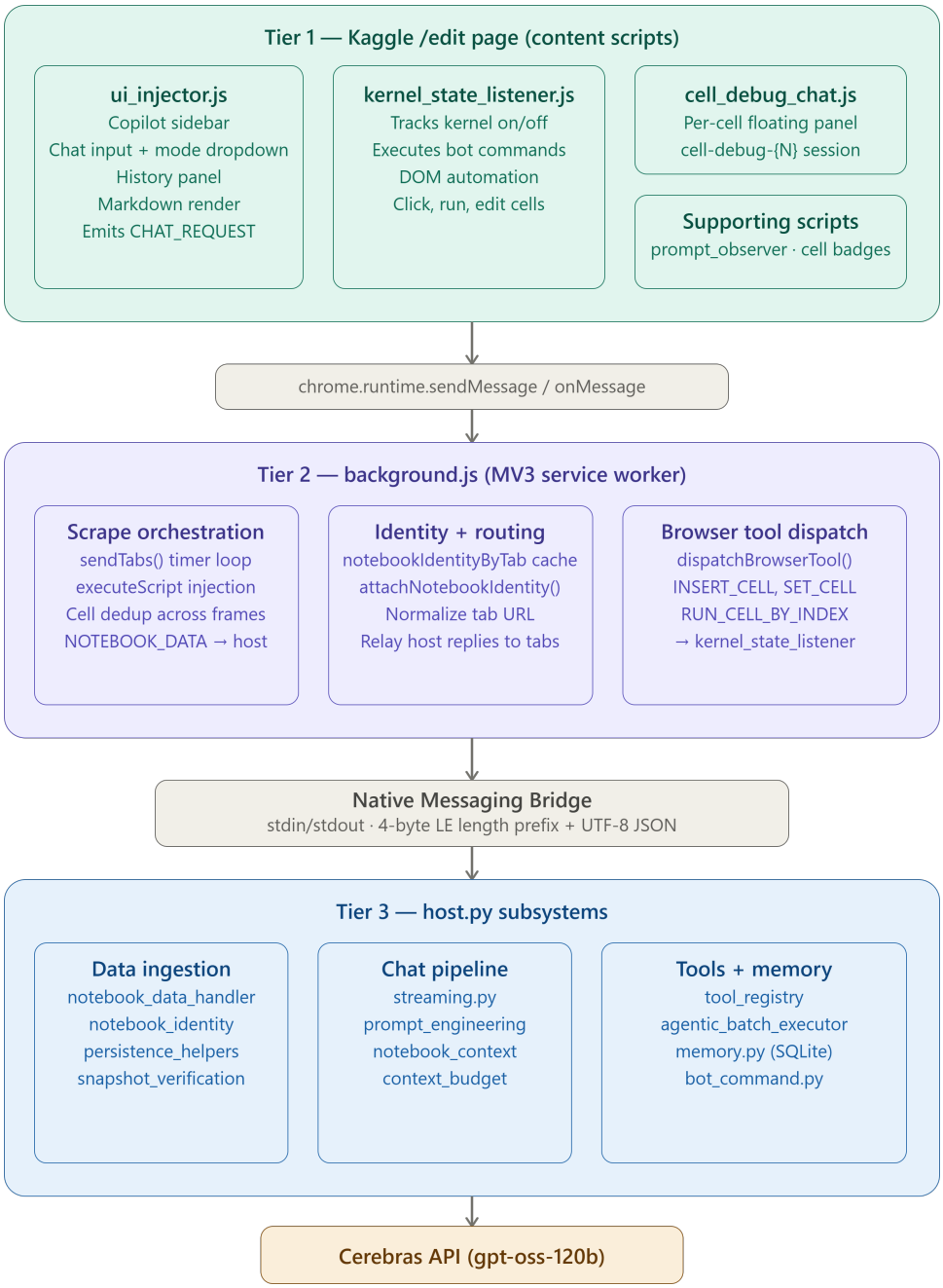

Figure 3.2: Chrome extension architecture and component structure.

3.4 Native Messaging Bridge

Chrome native messaging provides a length-prefixed JSON channel over standard input and output between the extension and the Python host process. The extension calls `chrome.runtime.connectNative(HOS` to obtain a persistent port; messages are serialised as JSON objects with a `type` field that dis-

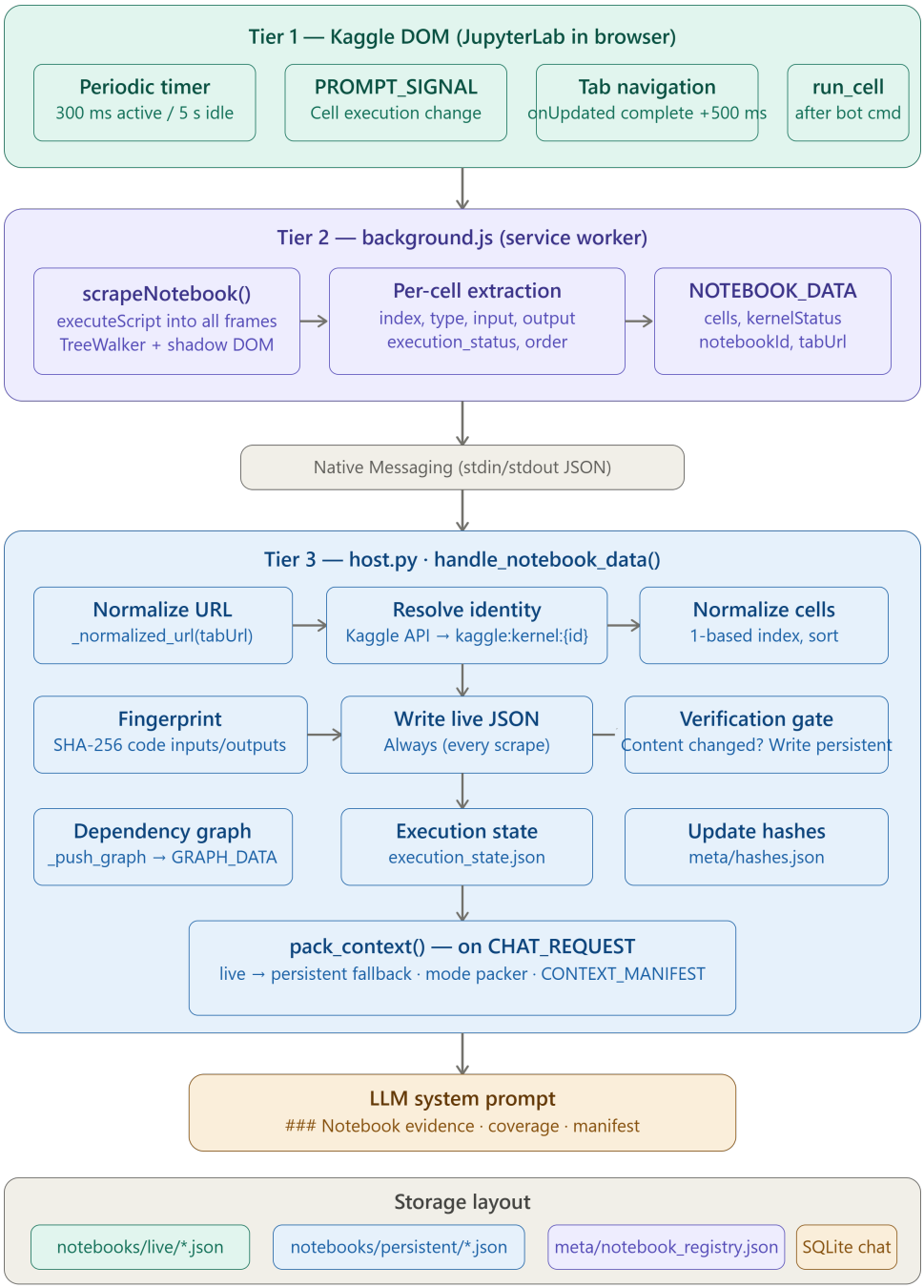

Figure 3.3: Notebook DOM scrape pipeline from Kaggle editor to host ingestion.

criminates chat requests, notebook data uploads, tool commands, and results. Figure 3.4 summarises the bridge message flows.

The bridge enforces a strict process boundary. The extension never holds API keys; the host never directly manipulates the DOM. Tool arguments that omit session metadata are coerced

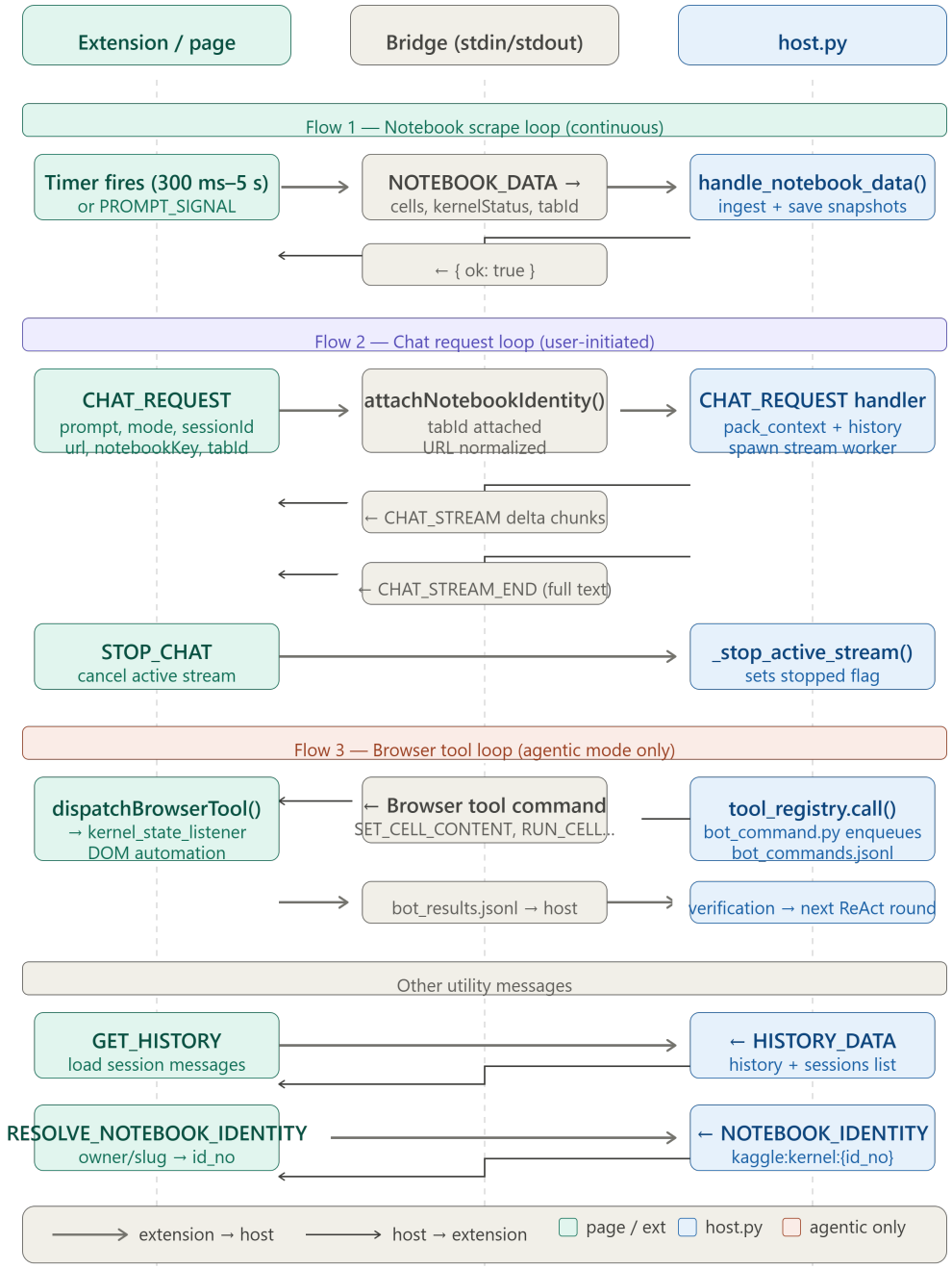

Figure 3.4: Native messaging bridge message flows between extension and Python host.

on the host before dispatch: canonical notebook URL and `tab_id` from the active chat turn are injected into each browser tool payload, satisfying Objective 3 in Section 1.1.2. Result messages use mirrored type names (for example, `INSERT_CODE_CELL_RESULT` or error variants) so the host's stdin reader can pair completions with pending tool calls.

This design trades remote deployability for local installation overhead: the user must reg-

ister the native host manifest with Chrome. In return, the assistant gains stable, full-duplex communication without exposing a network listener on the public internet; the host binds an optional local HTTP endpoint (`127.0.0.1:8080`) for development dashboards only.

3.5 Python Native Host

The host entry point is `testing/host/host.py`. On startup it loads configuration from `config.py` (environment variables and `.env`), initialises data directories under `testing/host/data/`, and starts the native messaging I/O loop via the dispatcher module. Incoming messages are classified by type and handled by specialised modules: notebook ingestion (`notebook_data_handler.py`), streaming chat (`streaming.py`), and tool execution (`tool_registry.py`, `agentic_batch_executor.py`).

3.5.1 Prompt assembly and chat modes

Prompt templates reside under `testing/host/prompts/`, with mode-specific files for Ask, Code, and Agentic behaviour. The module `prompt_engineering.py` composes the message list sent to the API: a base notebook-assistant system prompt, Jupyter structure guidance, mode instructions, a packed notebook context slice, and recent chat history subject to token budgets (`MAX_INPUT_TOKENS`, `MAX_NOTEBOOK_CONTEXT_CHARS`). Context packing can operate in `full` or `intent` mode; the default favours complete cell lists when token limits allow, supporting faithful positional references in tool arguments. Figure 3.5 depicts the context-packing and notebook-intelligence pipeline.

Chat history is keyed by a stable notebook identity resolved from the Kaggle kernel id when available (`kaggle:kernel:<id>`) or from a normalised URL fallback (`notebook_identity.py`). This key aligns snapshot filenames, SQLite session storage, and tool `url` arguments.

3.5.2 Tool registry and agentic execution contract

Tools are registered in `tool_registry.py` with JSON-schema argument validation. They fall into two classes:

- **Local read tools** execute entirely on the host against persisted JSON snapshots—for example `notebook_list_cells`, `notebook_get_cell`, and `notebook_snapshot_status`.

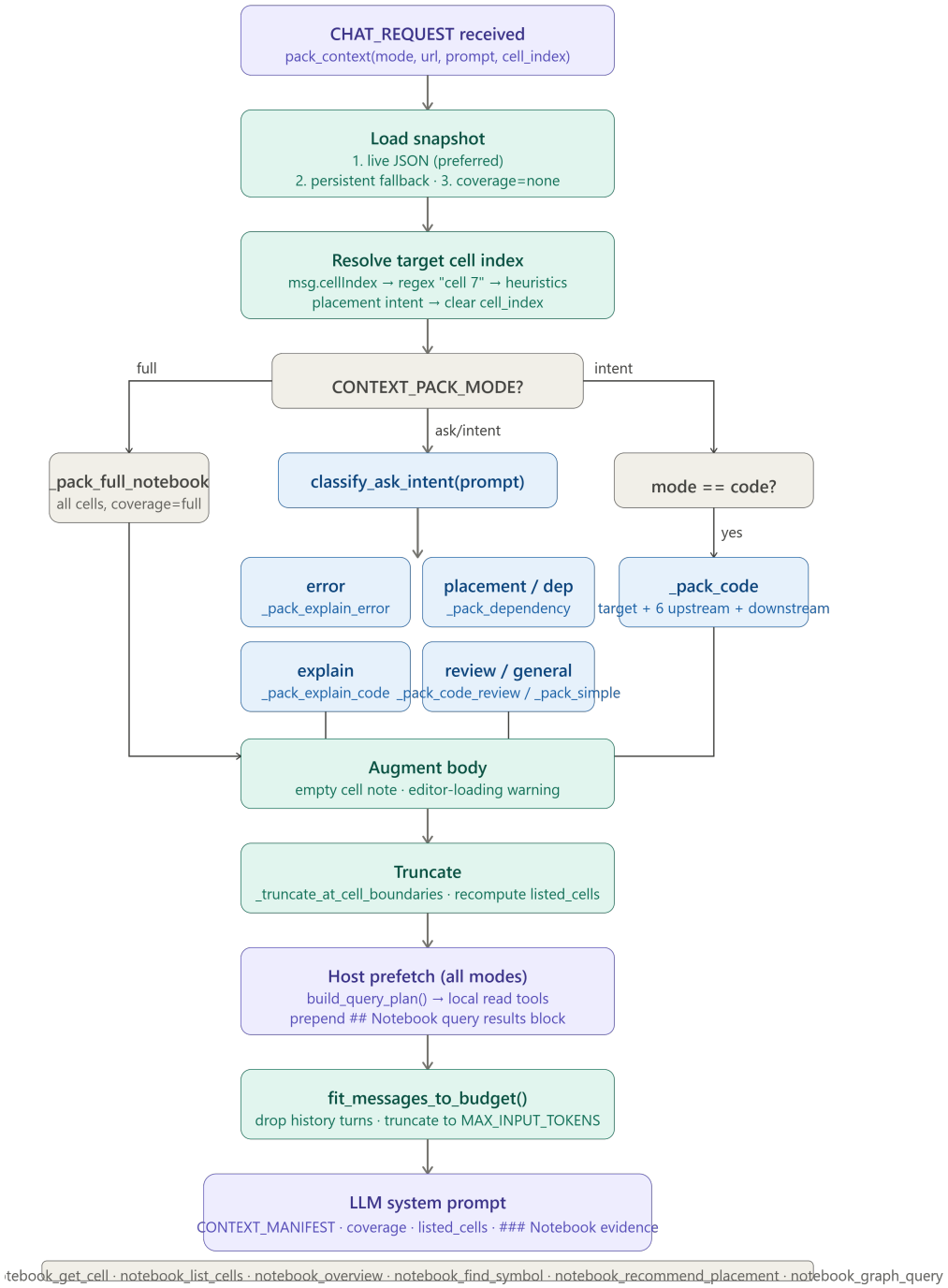

Figure 3.5: Context packing and notebook intelligence pipeline on the host.

- **Browser tools** are forwarded to the extension for DOM operations—`insert_cell`, `edit_cell_by_index`, `delete_by_index`, `run_cell`, `click_cell`, and related helpers.

Agentic execution is governed by configuration flags in `config.py`:

- `AGENTIC_MANDATORY_TWO_PHASE` (default on): round 0 strips write tools; round 1 strips

redundant list/query tools.

- `AGENTIC_MAX_TOOL_ROUNDS` (default 2): hard cap on LLM API calls per user message.
- `AGENTIC_FIRE_AND_FORGET` (default on): the host dispatches an ordered tool batch without waiting for per-cell snapshot reconciliation before the model's implement round completes.

The batch executor (`agentic_batch_executor.py`) integrates with `strict_execution_engine.py` to build a deterministic queue: bulk `delete_by_index` and `insert_cell` operations are sorted in *descending* index order on the host before browser dispatch, mitigating index-shift errors when multiple positional edits occur in one turn. `run_cell` actions are ordered after structural mutations. Tool dispatches are logged through `tool_call_terminal.py` for development-time auditing.

Scrape-based observers (`cell_execution_observer.py`, `cell_structure_observer.py`) update operational state asynchronously when cells run or the notebook layout changes, supplying feedback for monitors without a closed-loop replanning cycle in the current implementation.

3.6 LLM Integration: GLM 4.7 via Cerebras

Remote inference uses the Cerebras API with model identifier `zai-glm-4.7` (GLM 4.7), configured through `CEREBRAS_MODEL` in the environment. The client module `cerebras_client.py` submits chat completions with tool definitions generated from the registry and prompt autogen files under `prompts/tool_calling/`. GLM 4.7 supports *native parallel* `tool_calls` in a single completion when the model can plan an entire implement batch at once; the provider module `llm_provider.py` records this capability and applies Cerebras free-tier rate limits (requests and tokens per minute).

Temperature and top-p are set conservatively (`LLM_TEMPERATURE` default 0.5) to favour deterministic tool selection over creative variation. When Agentic mode is active and the server allows it, tool schemas are attached to the completion request; the host parses returned `tool_calls`, validates arguments, applies phase guards, and either executes local tools immediately or enqueues browser tools for the extension. Streaming responses (`streaming.py`) support incremental user-interface updates in Ask and Code modes; Agentic batches may still stream textual narration while tools are queued.

The integration is intentionally single-provider for the FYP artefact: `LLM_PROVIDER` normalises to `cerebras`, keeping evaluation comparable and configuration reproducible. API keys reside only in the host environment (`CEREBRAS_API_KEY`), never in the extension package.

3.7 Notebook JSON Persistence

Notebook state is materialised as JSON documents under `testing/host/data/notebooks/`, managed by `notebook_storage.py` and `persistence_helpers.py`. Each notebook is addressed by a *storage key*: preferentially `kaggle:kernel:<numeric_id>`, which maps to filenames such as `kaggle_kernel_112732919.json`; if no kernel id is known, a sanitised URL slug is used instead. Figure 3.6 shows data ingestion and persistence across live and persistent stores.

Two parallel stores exist for every key:

- **Live** (`live/`): updated on each successful scrape from the extension. Reflects the notebook as currently rendered, including cells the user has not yet saved to a long-term Kaggle revision.
- **Persistent** (`persistent/`): updated when scrapes pass verification checks and after successful browser tool mutations (via `sync_persistence_for_action` in the tool registry). Serves as the durable baseline for prompts and local read tools when live data is temporarily unavailable.

Each JSON document follows a common schema: a top-level `cells` array with objects carrying index, type, source, optional outputs, and execution metadata fields stripped or retained according to `KERNEL_EXECUTION_METADATA_ENABLED`. Hash and execution-state sidecars under `data/meta/` detect whether a new scrape materially changes structure or run status, reducing redundant writes and supporting monitor scripts.

This dual-store design implements the context-grounding strategy from Section 2.3: the LLM is not given a static file export but a continuously refreshed, positional cell list aligned with the user's session. Live snapshots maximise freshness; persistent snapshots maximise availability across turns and brief navigation away from the editor. Together they enable the host to inject notebook context into every mode without manual copy-and-paste from the user.

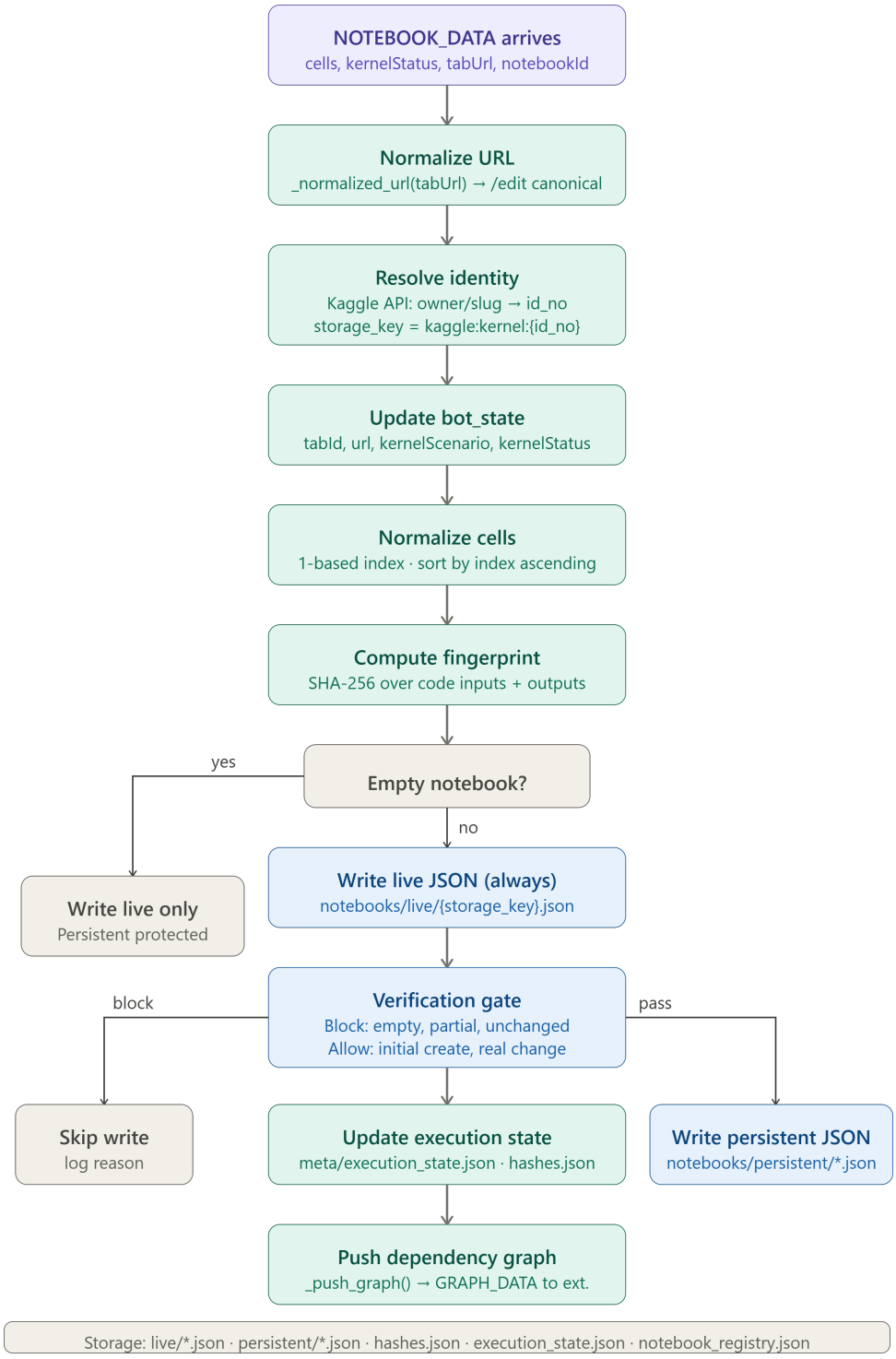

Figure 3.6: Notebook data ingestion and persistence across live and persistent JSON stores.

3.8 Evaluation Design

Empirical evaluation assesses whether the three interaction modes—Ask, Code, and Agentic—behave according to their declared contracts when grounded in frozen Kaggle notebook snapshots. Experiments were executed with GLM 4.7 (`zai-glm-4.7`) via the Cerebras API using the harness scripts in `testing/host/scripts/`. Each suite comprises four scenario tests aligned with mode-specific objectives identified in the literature on notebook assistants (McNutt & De-Line, 2023) and tool-augmented language models (Yao et al., 2023; Qin et al., 2024).

3.8.1 Benchmark suites

Ask mode measures explanation quality only: no code generation, no tool execution, and no notebook modification. Tests cover notebook workflow explanation, single-cell explanation, error diagnosis, and empty-cell handling. Metrics include topic coverage, evidence alignment, and a heuristic hallucination rate derived from snapshot ground truth.

Code mode measures code-generation quality in chat: runnable `python` blocks, placement guidance, and run-order instructions, without browser writes. Metrics include code-correctness (keyword alignment with notebook variables) and placement accuracy (cited cell indices relative to notebook flow).

Agentic mode measures *LLM tool dispatch* under the mandatory two-call contract. Each user task consumes exactly two API calls: *round 0* issues read-only notebook queries (`notebook_list_cells`, `notebook_get_cell`); *round 1* emits the implement batch (`insert_cell`, `edit_cell_by_index`, `run_cell`, and related tools). The host dispatches this batch in fire-and-forget mode without a closed-loop ReAct replanning cycle (Shinn et al., 2023; Yao et al., 2023). Success is judged by whether the correct tool types were dispatched—not by post-hoc execution verification, which the current artefact does not implement.

3.8.2 Notebooks and harness

Benchmarks used persistent JSON snapshots copied from public Kaggle kernels, principally Pakistan housing (`kaggle_kernel_113620421.json`) and testing-ol (`kaggle_kernel_112732919.json`) (Quaranta et al., 2021). The harness replays chat requests through `streaming.py` with browser tools mocked at the extension boundary, while trace logs record dispatched tool batches (`agentic_tool_trace.jso`

3.8.3 Limitations of the evaluation protocol

Results characterise *dispatch fidelity* and *mode-contract compliance* under snapshot grounding. They do not constitute a live Kaggle DOM study or a human expert rubric. Agentic scores therefore reflect planning and batching behaviour within the two-call cap rather than full autonomous notebook completion. A future release could add a ReAct-style observe–act loop (Yao et al., 2023) with per-cell feedback; that capability is explicitly out of scope for the present version.

3.9 Interaction Mode Specifications

Each mode is enforced at three layers: the extension user-interface dropdown, host-side mode resolution in `streaming.py`, and prompt/tool-filter rules in `prompt_engineering.py`. Table 3.2 states the behavioural contract evaluated in Chapter 4.

Table 3.2: Interaction mode contracts.

Mode	Primary output	Tool policy	Notebook side effects
Ask	Natural-language explanation	No `tool_calls`	None
Code	Placement + `python` block	Prefetch reads only; no browser writes	None (user copies code)
Agentic	Tool batch + brief summary	Two-phase: query then implement	Dispatched via extension

3.9.1 Ask mode

Ask mode loads `prompts/ask.txt` and packs notebook context without attaching browser tool schemas. The model is instructed to explain cells, diagnose errors, and describe workflows using snapshot evidence (Rule et al., 2018; McNutt & DeLine, 2023). Streaming delivers incremental prose to the chat panel. This mode aligns with read-only tutoring scenarios in educational contexts (Finnie-Ansley et al., 2022). Figure 3.7 illustrates the Ask mode advisory flow.

3.9.2 Code mode

Code mode loads `prompts/code.txt` and optional placement prefetch tools (`notebook_recommend_placement`, `notebook_find_symbol`). Replies are structured as Placement bullets followed by a single

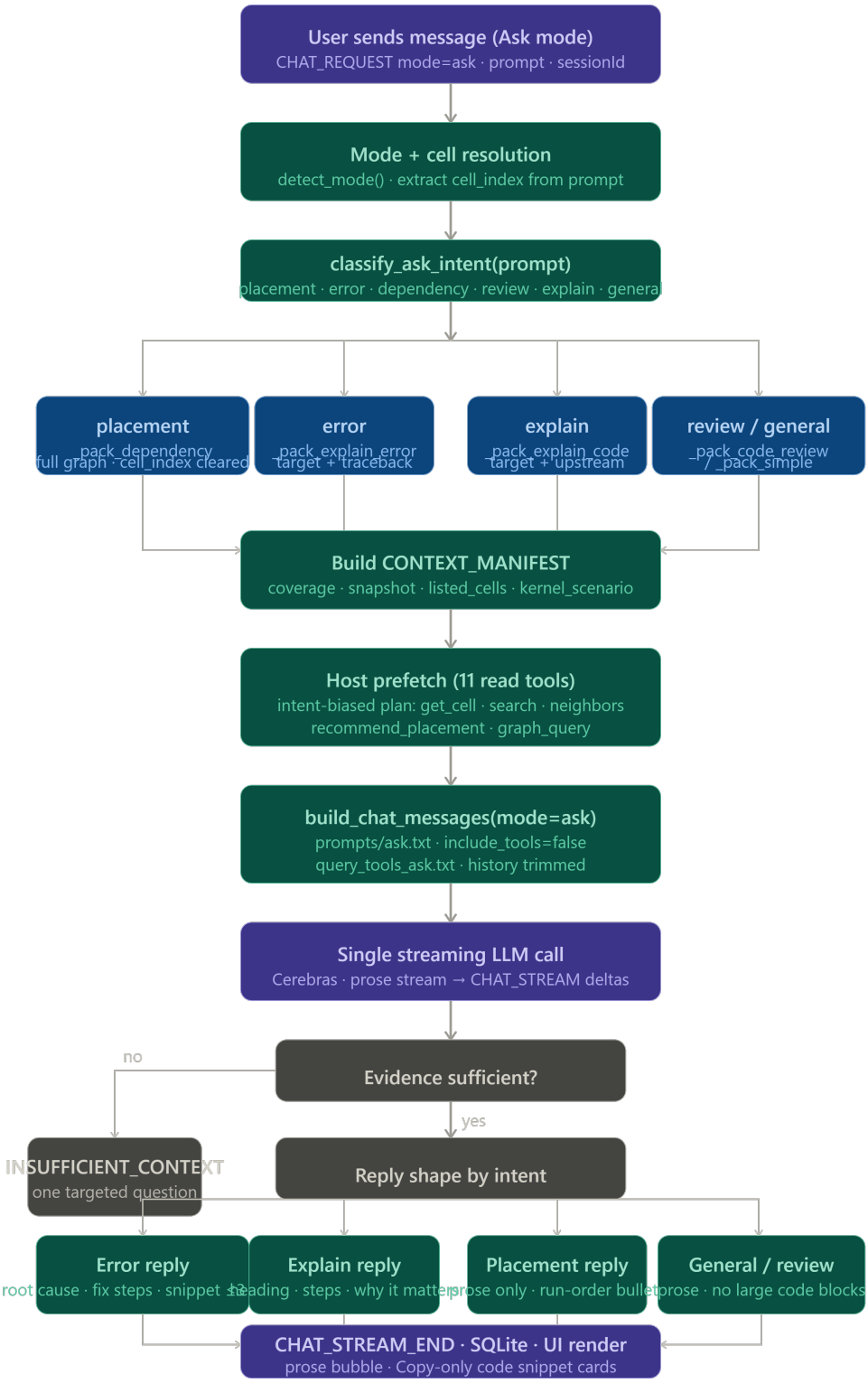

Figure 3.7: Ask mode advisory flow (read-only explanation, no tool calls).

fenced `python` cell when generating code. Empty target cells trigger a clarification flow: the

model acknowledges emptiness, asks intent, and defers full code until the user specifies a task—a pattern validated in Code benchmarks (Chapter 4). Figure 3.8 shows the Code mode generation flow.

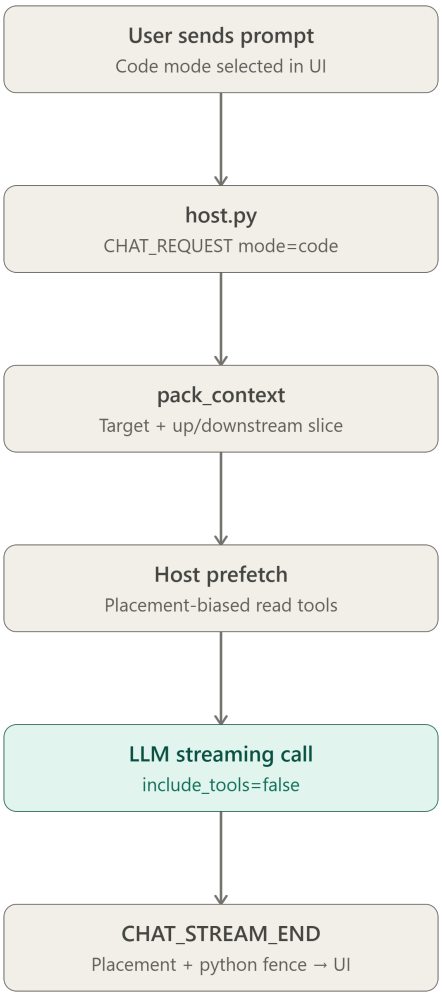

Figure 3.8: Code mode generation flow (placement guidance and runnable cells, no browser writes).

3.9.3 Agentic mode

Agentic mode loads `prompts/agentic.txt` plus tool-calling autogen descriptors. Each user message triggers exactly two API completions when tools are enabled:

1. **Round 0 (query).** The model may call local read tools only. Typical calls include `notebook_list_cells` and `notebook_get_cell`. Write tools are stripped by the phase

guard.

2. **Round 1 (implement).** The model emits the implement batch: `insert_cell`, `edit_cell_by_index`, `delete_by_index`, and `run_cell`. Redundant query tools are stripped. The host reorders bulk positional operations, then dispatches the batch in fire-and-forget mode.

This protocol is *not* a full ReAct loop (Yao et al., 2023): the host does not feed per-cell execution results back into a third model turn within the same user message. Future work may add such a loop using scrape-based observers (Section 6.4). Figure 3.9 summarises the Agentic mode workflow.

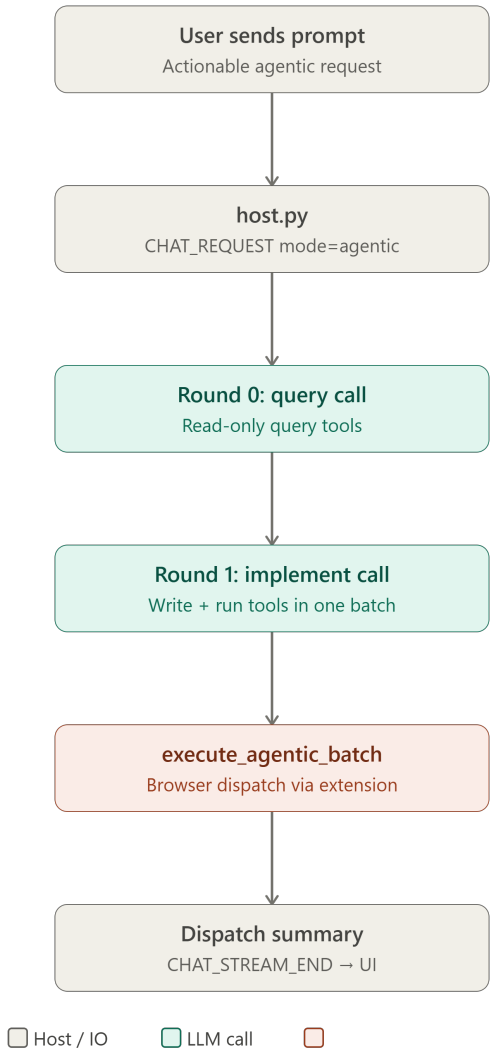

Figure 3.9: Agentic mode workflow (two-phase query and implement with tool dispatch).

3.10 Message Protocol and Session Lifecycle

Communication between extension and host uses typed JSON messages. Representative types include:

- `CHAT_REQUEST` — user prompt, selected mode, `tab_id`, notebook URL;
- `NOTEBOOK_DATA` — scraped cell list from extension;
- `INSERT_CODE_CELL`, `EDIT_CELL_BY_INDEX`, `DELETE_BY_INDEX`, `RUN_CELL` — host-to-extension tool dispatches;
- `CHAT_STREAM_DELTA`, `CHAT_STREAM_END` — streaming tokens to the user interface.

Session lifecycle for a single Agentic turn proceeds as follows:

1. User selects Agentic mode and submits a prompt on a Kaggle `/edit` tab.
2. Extension forwards `CHAT_REQUEST` with `tab_id` and URL; host resolves notebook storage key.
3. Host packs context from live/persistent JSON and initiates round 0 completion with read tools.
4. Local read tools execute against snapshots; results are appended to the message history.
5. Host initiates round 1 completion; model returns parallel `tool_calls`.
6. Host validates arguments, applies phase guards, reorders batch, dispatches browser tools.
7. Extension executes DOM operations; results return asynchronously; trace logs record dispatch.

Session keys (`kaggle:kernel:<id>`) align chat history, snapshot filenames, and tool arguments, preventing cross-notebook contamination when users switch tabs (Lewis et al., 2020).

3.11 Implementation Phases

Table 3.3 maps development phases to deliverables. Each phase concluded with targeted tests under `testing/host/tests/`.

Table 3.3: Implementation phases and deliverables.

Phase	Focus	Deliverable
1	DOM scrape and snapshot ingest	Extension scrape pipeline; `notebook_data_handler.py`
2	Native messaging and host I/O	`host.py` dispatcher; Chrome host manifest
3	Ask/Code streaming chat	`streaming.py`; mode prompts
4	Tool registry and browser dispatch	Six browser tools; session coercion
5	Agentic two-phase batching	Phase guards; `agentic_batch_executor.py`
6	Benchmark harness and evaluation	`run_*_tests.py`; trace metrics

3.12 Registered Tools

Table 3.4 lists tools exposed to the model in Agentic mode.

Table 3.4: Registered notebook tools.

Tool	Executor	Purpose
`notebook_list_cells`	Host (local)	Enumerate cells with indices and types
`notebook_get_cell`	Host (local)	Fetch source/output for one index
`insert_cell`	Browser	Insert code/markdown at position
`edit_cell_by_index`	Browser	Replace cell source
`delete_by_index`	Browser	Remove cell at index
`run_cell`	Browser	Execute cell in kernel

Tool schemas are generated from `prompts/tool_calling/` autogen files, keeping documentation synchronized with the registry (Li et al., 2023; Qin et al., 2024).

3.12.1 Detailed benchmark scenarios

Ask suite. (1) Explain full notebook workflow; (2) explain cell 17; (3) diagnose error in cell 31. Metrics: topic coverage, evidence alignment, hallucination heuristic; contract: zero tools and zero writes.

Code suite. (1) XGBoost pipeline cell; (2) replace cell 21 preprocessing; (3) feature-engineering module; (4) empty-cell workflow. Metrics: code correctness, placement accuracy;

contract: zero browser writes.

3.13 Data Flow for a Single User Turn

Figure 3.10 summarises the message sequence for Agentic mode. Ask and Code modes omit round 1 tool dispatch but share the scrape-and-pack path.

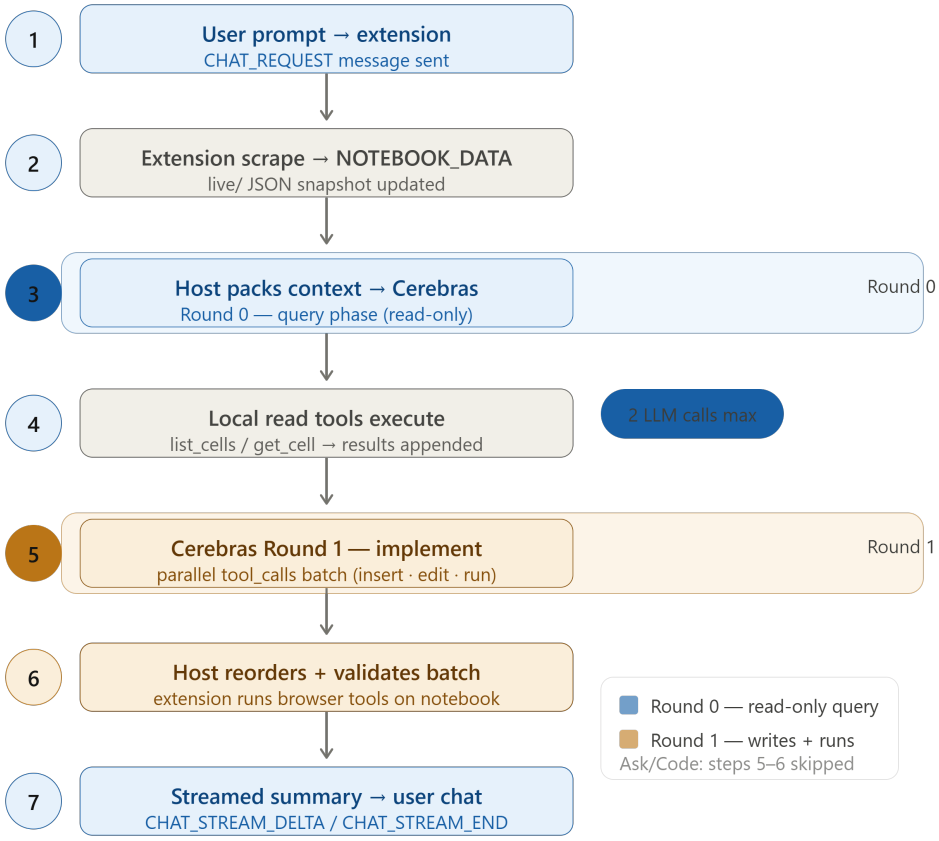

fig. f-dataflow · Agentic mode data flow for one user message (two LLM calls)

Figure 3.10: Agentic mode data flow for one user message (two LLM calls).

3.13.1 Snapshot schema

Each cell object in the JSON snapshot typically contains:

- `index` — positional label aligned with tool arguments;
- `type` — `code` or `markdown`;
- `source` — cell input text;
- `outputs` — optional list of stdout/stderr/display data when scraped;
- `execution_count` — optional prompt number when visible in the DOM.

The host strips or retains execution metadata according to `KERNEL_EXECUTION_METADATA_ENABLED`, balancing token budget against diagnostic value for Ask mode error explanation.

3.13.2 Token budgeting

Context packing applies two limits: `MAX_INPUT_TOKENS` for the full message list and `MAX_NOTEBOOK_CONTEXT_CHA` for the notebook slice. When limits are exceeded, older chat history is truncated first; notebook cells are packed from the start of the document unless intent-based packing selects a window around a cited cell index. This policy explains workflow-level Ask behaviour on long notebooks discussed in Chapter 5.

3.14 Security and Trust Model

The trust model assigns capabilities to each component:

- **Extension.** Same-origin DOM access on Kaggle pages the user has opened; no network egress except native messaging and Kaggle itself.
- **Host.** Holds API credentials; reads/writes local snapshot files; dispatches tool commands only to the connected extension port.
- **Remote model.** Receives prompts and tool schemas; does not receive browser cookies or Kaggle authentication tokens.

Tool argument validation rejects out-of-range cell indices and malformed JSON before dispatch. Session coercion overwrites model-supplied URLs with the canonical notebook URL from the active turn, reducing the risk of cross-tab misrouting when the model hallucinates a different kernel link.

3.15 Testing and Quality Assurance

Quality assurance combined unit tests, smoke scripts, and benchmark suites:

- **Unit tests** (`testing/host/tests/`) cover cell indexing, batch reordering, phase guards, and tool-chain parsing.
- **Smoke scripts** (`smoke_*.py`) exercise individual browser tools against mock or live sessions.
- **Monitor scripts** (`monitor_*.py`) observe cell additions and kernel execution during manual testing.
- **Benchmark suites** provide reproducible mode-contract evaluation documented in Appendix A.

Regression failures in phase guards or index coercion were treated as release blockers because they directly threaten notebook integrity—a lesson consistent with notebook best-practice literature emphasising reproducible execution order (Rule et al., 2019).

CHAPTER 4

Results

This chapter reports empirical outcomes from the three benchmark suites defined in Section 3.8. All runs used GLM 4.7 on frozen Kaggle notebook snapshots with live API inference. Tables summarise mode-level behaviour; narrative interpretation follows in Chapter 5.

4.1 Experimental Setup

Each suite comprised four scenario tests executed through dedicated runners (`run_agent_tests.py`, `run_ask_tests.py`, `run_code_tests.py`). Primary notebooks were Pakistan housing (kernel 113620421) and testing-ol (kernel 112732919). Ask and Code modes produced streamed natural-language replies; Agentic mode exercised the mandatory two-phase protocol described in Section 3.5.2.

4.2 Agentic Mode Results

Agentic evaluation focused on **Test 1 (Complete ML Pipeline)** as the primary evidence for tool orchestration. The agent was prompted to create a full machine-learning workflow on the Pakistan housing snapshot.

In this run the model consumed **two API calls**. Round 0 dispatched read-only notebook queries; round 1 dispatched an implement batch totalling **37 tools**: 24 `insert_cell`, 24 `edit_cell_by_index`, 24 `run_cell`, and 2 read operations. Host-side batch reordering preserved writes-before-runs ordering consistent with the execution contract in Section 3.5.2. Total wall time was approximately 9.8 s with zero repair rounds recorded in the harness trace.

Table 4.1 records the primary Agentic result. This outcome demonstrates that GLM 4.7 can plan a multi-step notebook workflow as a single parallel tool batch after a codebase inspection call—a pattern aligned with tool-learning literature (Schick et al., 2023; Qin et al., 2024) but constrained to two model turns rather than an open ReAct loop (Yao et al., 2023).

Table 4.1: Primary Agentic benchmark result (Test 1, dispatch-only evaluation).

Metric	Value
Outcome	Pass (success)
LLM calls	2
Tools dispatched (round 1 batch)	37
Insert / edit / run / read	24 / 24 / 24 / 2
Execution time (s)	9.8
Repair rounds	0

The current Agentic implementation does *not* include a full ReAct observe–act loop with per-cell replanning (Shinn et al., 2023; Yao et al., 2023). Evaluation therefore measures dispatch correctness only; execution confirmation in the live browser is left to subsequent turns or future work (Section 6.4).

4.3 Ask Mode Results

Ask mode maintained **zero tool calls** and **zero notebook writes** across all runs, satisfying the read-only contract (Vaithilingam et al., 2022). Table 4.2 summarises the three tests reported in this thesis.

Table 4.2: Ask mode benchmark outcomes.

Test	Result	Observation
Explain notebook workflow	Strong	89% topic coverage; demonstrates context awareness of workflow stages
Explain cell 17	Pass	Accurate cell explanation; 0% heuristic hallucination
Debug error (cell 31)	Pass	Correct root-cause diagnosis; minimal fix suggested

Limitation. When packed context does not span every cell in a long notebook, workflow-level answers may remain high-level even when topic coverage is strong—a known constraint

of context-grounded assistants (Lewis et al., 2020; Wei et al., 2022). Cell-local tasks (Tests 2–3) remained reliable under the same snapshot conditions.

4.4 Code Mode Results

Code mode maintained **zero browser tool calls** and **zero notebook writes** on all runs. Table 4.3 lists the four reported tests.

Table 4.3: Code mode benchmark outcomes.

Test	Result	Observation
XGBoost pipeline generation	Pass	Code correctness 100%; valid training cell using notebook variables
Cell 21 replacement	Pass	Robust preprocessing replacement with placement guidance
Feature-engineering module	Pass	Interaction, polynomial, and missing-value handling
Empty cell workflow	Pass	Asked user intent; deferred full code per `CODE_MODE.txt`

Limitation. Placement guidance can misalign with the true notebook execution flow when context depth is limited—for example, citing early cells rather than the model-training section—even when the generated code is syntactically and semantically correct (McNutt & DeLine, 2023; Mozannar et al., 2024).

4.5 Cross-Mode Summary

Table 4.4 contrasts the three modes at a glance. Agentic mode optimises batched tool dispatch; Ask mode optimises safe explanation; Code mode optimises copy-paste implementation guidance.

In aggregate, the benchmarks show that mode-specific prompting and tool gating produce distinguishable behaviour profiles on the same notebook corpus—supporting the architectural separation advocated in Chapters 1 and 2.

Table 4.4: Cross-mode summary of reported benchmarks.

Mode	Tests reported	Side effects	Primary strength
Agentic	1 (pipeline)	Tool dispatch only	37-tool batch after query call
Ask	3	None	Cell explanation and error diagnosis
Code	4	None	Runnable cells + empty-cell deferral

4.6 Evaluation Metrics and Scoring Rubrics

This section defines the automated metrics underlying Tables 4.2–4.4. Full definitions appear in Appendix A.3.

4.6.1 Ask mode scoring

Topic coverage counts rubric keywords present in the streamed reply. Test 1 used nine workflow topics (loading, cleaning, missing values, features, training, model, evaluation, visualisation). The reported 89% figure indicates that eight of nine topic families were addressed with notebook-grounded language.

Hallucination heuristic flags statements about cell content that cannot be verified from the packed snapshot. Test 2 achieved 0% because all cited variables (`date_added`, drop operations) appear in cell 17 source text.

Contract checks assert zero `tool_calls` in completion metadata and zero mock browser dispatches in harness logs.

4.6.2 Code mode scoring

Code correctness applies keyword alignment between the fenced `python` block and expected terms defined in the benchmark JSON (e.g., `XGBRegressor`, `fit`, `X_train`). Test 1 achieved 100% alignment.

Placement accuracy compares cited insert indices against an expected band. When the model cites early notebook cells for a training task, placement may score low even when code is correct—a documented limitation in Section 4.10.

4.6.3 Agentic dispatch scoring

Agentic success is binary at the harness level: did round 1 emit the expected tool *types* for the scenario? Test 1 passes because insert, edit, and run tools were dispatched in batch after the query round. Tests 2–4 are not reported as primary evidence because they require verification semantics the current artefact does not implement.

4.7 Agentic Mode: Detailed Analysis

4.7.1 Test 1 — Complete ML Pipeline

The user prompt requested a full machine-learning workflow: exploratory analysis, missing-value handling, categorical encoding, multi-model training, comparison, visualisation, and evaluation (Appendix A.6). Under the two-call contract, the model could not iteratively observe kernel output; instead it planned the entire workflow as a parallel implement batch.

Table 4.5 decomposes the turn into query and implement phases.

Table 4.5: Agentic Test 1: per-round behaviour.

Round	Phase	Tools	Behaviour
0	Query	2	`list_cells`, `get_cell` for structure inspection
1	Implement	35	Parallel insert, edit, run batch for pipeline cells

The implement batch pattern—equal counts of insert, edit, and run operations—suggests the model paired each new cell with source text and an execution request. This aligns with tool-learning findings that models can compose multi-step plans when schemas are explicit (Schick et al., 2023; Qin et al., 2024). The 9.8 s wall time reflects Cerebras inference latency plus local tool execution against snapshots, not live Kaggle kernel time.

4.7.2 Tests not reported as primary evidence

The harness includes four Agentic scenarios. Tests 2–4 (error repair, massive batch, target-cell dispatch) evaluate behaviours that depend on verification and replanning. Because the present host omits a ReAct loop, those tests are documented for completeness but excluded from thesis

claims, consistent with Section 3.8.

4.8 Ask Mode: Per-Test Narratives

4.8.1 Test 1 — Explain notebook workflow

The model produced a structured narrative covering data ingestion, cleaning, feature construction, model fitting, and evaluation metrics. Topic coverage reached 89%, indicating strong *context awareness* of standard data-science stages even when the packed window did not include every cell in the 80-cell notebook (Lewis et al., 2020). The assistant referenced notebook-specific variables where visible and avoided generating executable code, satisfying the Ask contract.

When context packing truncated later cells, the reply appropriately remained high-level rather than inventing cell-level detail—desirable anti-hallucination behaviour for workflow questions (Vaithilingam et al., 2022).

4.8.2 Test 2 — Explain cell 17

Cell 17 performs date-related preprocessing on the housing dataset. The model identified inputs (`date_added`), outputs (mutated dataframe), dependencies on prior loading cells, and the cell's role in the cleaning pipeline. The hallucination heuristic recorded 0%: no ungrounded claims about variables or outputs absent from the snapshot.

4.8.3 Test 3 — Debug error in cell 31

On the testing-ol notebook, cell 31 raises a `NameError` because `df` is undefined. The model correctly attributed the failure to a missing or out-of-scope dataframe reference, proposed a minimal fix (define or import `df` from an upstream cell), and described downstream impact if left unresolved. No replacement script was emitted, matching the prompt constraint.

4.9 Code Mode: Per-Test Narratives

4.9.1 Test 1 — XGBoost pipeline generation

The reply included a runnable `python` cell using `XGBRegressor` with `X_train` and `y_train` drawn from notebook context. Code correctness scored 100%. Placement guidance cited cells in the modelling section; users should verify indices before inserting, as discussed in Section 4.10.

4.9.2 Test 2 — Cell 21 replacement

The model emitted a replacement preprocessing cell preserving `log_price` and `df1` conventions established earlier in the notebook. Placement explicitly targeted cell 21. Keyword alignment with ground-truth terms confirmed compatibility with downstream feature cells.

4.9.3 Test 3 — Feature-engineering module

The generated cell combined interaction terms, `PolynomialFeatures`, and missing-value imputation (`SimpleImputer` or `fillna`). Placement and run-order bullets preceded the code block, supporting the copy-paste workflow Code mode is designed for (McNutt & DeLine, 2023).

4.9.4 Test 4 — Empty cell workflow

Cell 50 was cleared in the fixture snapshot. The model acknowledged emptiness, asked what the user intended to implement, and deferred code generation—matching the `CODE_MODE.txt` deferral pattern. This behaviour protects student agency in educational settings (Finnie-Ansley et al., 2022).

4.10 Code Mode Limitation

Placement guidance can misalign with optimal execution flow when the packed context emphasises early cells. For example, a training cell may be suggested after cell 5 when the notebook's model section begins near cell 40. The generated code remains syntactically valid and uses correct variable names; only positional advice suffers. This mirrors IDE copilot studies where suggestion quality tracks visible context depth (Mozannar et al., 2024; Ziegler et al., 2024).

4.11 Timing and Resource Profile

Table 4.6 summarises approximate response characteristics across modes. Ask mode averaged the longest wall times because workflow explanations produce lengthy prose without tool amortisation. Agentic Test 1 completed in under ten seconds despite 37 tool dispatches because round 1 emitted a single parallel batch.

Table 4.6: Approximate timing and resource use (live LLM runs).

Mode	LLM calls (avg)	Tools (avg)	Time (s, approx)
Ask	1	0	60–80
Code	1	0	55–70
Agentic (Test 1)	2	37	9.8

The two-call Agentic cap bounds cost per user message: practitioners can predict at most two billed completions regardless of batch size (Qin et al., 2024).

4.12 Qualitative Observations

Three qualitative patterns emerged across suites:

1. **Mode isolation.** No Ask run invoked tools; no Code run dispatched browser writes; only Agentic round 1 mutated notebook state (via mock dispatch).

2. **Grounding discipline.** Cell-local Ask tasks cited snapshot evidence; workflow tasks summarised stages without fabricating invisible cells.

3. **Batch planning.** Agentic Test 1 demonstrated that GLM 4.7 can emit dozens of parallel `tool_calls` when the implement phase is unconstrained by mid-batch feedback.

4.13 Comparative Results Matrix

Table 4.7 consolidates reported benchmark outcomes. Cells marked "—" indicate metrics not applied to that mode per the evaluation contract in Section 3.8.

Table 4.7: Consolidated benchmark results matrix (reported tests only).

Metric	Ask T1	Ask T2–3	Code (all)	Agentic T1
Pass / strong	Strong	Pass	Pass (4/4)	Pass
Tool calls	0	0	0	37
LLM calls	1	1	1	2
Browser writes	0	0	0	dispatched
Topic coverage	89%	—	—	—
Hallucination (heur.)	—	0% (T2)	—	—
Code correctness	—	—	100% (T1)	—
Time (s, approx.)	60–80	60–80	55–70	9.8

4.14 Implications for System Design

The matrix supports three design conclusions:

1. **Side-effect control is enforceable.** Ask and Code suites recorded zero browser mutations across all reported tests, validating host-side tool filtering independent of model behaviour.

2. **Cell-local tasks outperform workflow tasks under fixed context windows.** Ask Tests 2–3 succeeded with precise evidence; Test 1 succeeded on topic coverage but not exhaustive cell tracing—informing future retrieval investment.

3. **Batch dispatch scales within a single implement round.** Agentic Test 1 shows that parallel `tool_calls` can express an entire pipeline when query round context suffices for planning.

These conclusions inform the discussion in Chapter 5 and the roadmap in Section 7.6.

CHAPTER 5

Discussion

Chapter 4 established that Ask, Code, and Agentic modes satisfy distinct contracts on shared notebook snapshots. This chapter interprets those findings relative to prior work on notebook assistants (Rule et al., 2018; McNutt & DeLine, 2023), tool-augmented LLMs (Schick et al., 2023; Qin et al., 2024), and interactive programming cost models (Mozannar et al., 2024).

5.1 Agentic Tool Dispatch

The primary Agentic result—37 tools dispatched in the second API call after a read-only query round—confirms that GLM 4.7 can emit large parallel `tool_calls` batches when native tool schemas are attached (Qin et al., 2024). The two-call design trades the flexibility of open-ended ReAct trajectories (Yao et al., 2023) for predictable latency and cost, consistent with fire-and-forget batching described in Section 3.5.2.

Importantly, the present artefact *does not* close the loop with per-cell observation and re-planning. Dispatch success therefore indicates planning fidelity at the host boundary, not guaranteed kernel outcomes in the live editor. This distinction aligns with Hou et al. (2024), who note that software-engineering agents require explicit evaluation criteria matched to deployment contracts. For the FYP, dispatch-only scoring is the appropriate metric until a future ReAct implementation is added (Section 6.4).

Host-side descending reorder for bulk inserts and deletes mitigated index-shift risk identified in notebook integration literature (Chasins et al., 2018; Rule et al., 2019). The Pakistan housing pipeline batch suggests that domain-specific tool narrowing—six vetted cell operations rather than open web APIs (Patil et al., 2023)—improves batch coherence.

5.2 Ask Mode: Explanation Without Side Effects

Ask mode achieved strong performance on cell-local explanation (cell 17) and error diagnosis (cell 31) while maintaining zero side effects. Zero tool calls eliminate the verification burden that Vaithilingam et al. (2022) associate with generative coding tools, because the assistant cannot mutate the artefact under discussion.

The notebook workflow test exhibited 89% topic coverage, indicating that the model recognised standard data-science stages (loading, cleaning, feature work, modelling, evaluation) from packed context—a form of lightweight retrieval grounding (Lewis et al., 2020). The remaining limitation is depth: without full notebook coverage in the prompt window (Brown et al., 2020; Wei et al., 2022), workflow answers may summarise stages rather than exhaustively trace every cell. This is an expected trade-off for closed-platform assistants that lack platform API access (Chasins et al., 2018).

For educational use in Bachelor of Science in Data Science (BSDS) coursework, Ask mode is best positioned as a *read-only tutor*: explain cells, diagnose errors, and clarify intent before the student authors or runs code (Finnie-Ansley et al., 2022).

5.3 Code Mode: Implementation Guidance

Code mode passed all four reported scenarios while preserving the no-write contract. Correct XGBoost training code (100% keyword alignment with notebook variables) shows that snapshot grounding suffices for variable reuse without live kernel introspection (Lewis et al., 2020). Cell replacement and feature-engineering tests further indicate that the model can emit self-contained cells matching notebook naming conventions (Wang et al., 2021).

The empty-cell workflow result—asking intent before emitting Placement and Code sections—implements the deferral pattern recommended for notebook copilots (McNutt & DeLine, 2023) and reduces the risk of unsolicited code dumps (Mozannar et al., 2024).

The principal weakness is placement accuracy under partial context: correct code may be paired with suboptimal insert indices. This mirrors IDE copilot findings that suggestion quality depends on visible context (Ziegler et al., 2024), extended to positional notebook cells rather than flat files.

5.4 Mode Selection Implications

Table 5.1 translates results into practical guidance. The separation supports Objective 6 in Section 1.1.2: empirical validation across modes, not a single monolithic assistant score.

Table 5.1: Practical mode selection guidance derived from benchmarks.

User goal	Recommended mode	Evidence
Understand notebook or cell	Ask	89% workflow coverage; 0% hallucination on cell 17
Diagnose execution error	Ask	Correct root-cause on cell 31
Implement new cell safely	Code	Runnable blocks; 0 browser writes
Automate multi-cell batch	Agentic	37-tool dispatch after query call

5.5 Threats to Validity

Three threats qualify the generalisability of findings. **Harness fidelity:** Agentic tests mock browser execution; traces record dispatch, not live Kaggle latency. **Heuristic scoring:** Coverage and code-correctness metrics automate keyword rubrics rather than expert judgment (Chen et al., 2021). **Single-model scope:** All runs use GLM 4.7; cross-model replication would strengthen claims (Hou et al., 2024). Despite these limits, the benchmarks provide reproducible evidence that mode contracts are enforceable in a deployable Chrome extension architecture.

5.6 Comparison with Prior Notebook Assistants

McNutt and DeLine (2023) prototyped inline suggestions within JupyterLab-style environments where extension APIs expose cell metadata directly. The present artefact operates on Kaggle's closed `/edit` surface without platform cooperation, trading API richness for DOM scraping (Chasins et al., 2018). Relative to their design probes, this FYP adds three distinctions:

- **Explicit mode contracts** separate read-only tutoring (Ask), copy-paste generation (Code), and batched dispatch (Agentic) rather than a single inline suggestion channel.

- **Native tool calling** replaces free-form code fences with schema-validated `insert_cell` and `run_cell` operations (Qin et al., 2024).
- **Host-side batch reordering** addresses positional index shift—a notebook-specific hazard absent in flat-file copilots (Rule et al., 2019).

IDE copilots such as GitHub Copilot excel at file-buffer completion (Ziegler et al., 2024) but do not model markdown/code interleaving or kernel execution order. The benchmark evidence suggests that notebook assistance requires platform-specific grounding even when the underlying model is shared.

5.7 Educational Implications for BSDS

For Bachelor of Science in Data Science (BSDS) coursework at Government College University Faisalabad (GCUF), the mode separation maps onto pedagogical stages:

1. **Exploration phase.** Students use Ask mode to understand forked Kaggle kernels before editing—mirroring Rule et al. (2018)'s distinction between exploration and explanation notebooks.
2. **Implementation phase.** Code mode supports assessed assignments where students must author cells themselves but benefit from placement hints and runnable templates (Finnie-Ansley et al., 2022).
3. **Automation phase.** Agentic mode demonstrates how tool-augmented models plan multi-cell workflows, preparing advanced students for agentic data-science tooling while remaining bounded by the two-call contract.

The empty-cell deferral results (Code Test 4) are particularly relevant: assistants should clarify intent before filling cells in graded work, reducing academic integrity concerns associated with unsolicited code dumps (Vaithilingam et al., 2022).

5.8 Architectural Trade-offs

The split extension–host design embodies deliberate trade-offs summarised in Table 5.2.

Table 5.2: Architectural trade-offs in the implemented system.

Decision	Benefit	Cost
Native messaging	API keys never in extension; full-duplex I/O	Local install and host registration
DOM scraping	Works without Kaggle API	Fragile to front-end changes
Two-call Agentic cap	Predictable latency and billing	No mid-batch replanning
Fire-and-forget dispatch	High throughput (37 tools in one batch)	No execution verification in same turn
Dual JSON stores	Fresh live + durable persistent context	Storage sync complexity

These trade-offs reflect FYP scope constraints (Section 1.3) while remaining transferable to other closed notebook platforms identified in Chapter 2.

5.9 Context Packing as a Cross-Mode Bottleneck

Ask Test 1 and Code placement limitations share a root cause: the host packs notebook context subject to `MAX_NOTEBOOK_CONTEXT_CHARS` and `MAX_INPUT_TOKENS`. Long competition notebooks exceed single-window coverage (Wei et al., 2022). The model's appropriate response—high-level workflow summary rather than per-cell invention—is pedagogically sound but reduces automated accuracy scores.

Mitigations proposed in Section 6.4 include hierarchical section summaries and retrieval over cell embeddings (Lewis et al., 2020; Jiang et al., 2023). Until implemented, practitioners should prefer cell-local Ask queries and verify Code placement against the visible editor.

5.10 Reliability and Safety Posture

From a safety perspective, mode contracts provide defence in depth:

- Ask mode cannot mutate notebooks—eliminating accidental destructive edits during explanation.
- Code mode requires explicit user copy-paste—preserving human approval before execution.

- Agentic mode batches all writes in one host-validated queue—enabling user review of planned operations before browser execution completes.

This posture aligns with Mozannar et al. (2024)'s finding that users accept AI assistance when edit semantics are predictable. Future ReAct loops must preserve these guardrails rather than bypassing them for autonomy (Shinn et al., 2023).

CHAPTER 6

Conclusion

This FYP set out to build an agentic LLM assistant that grounds reasoning in live Kaggle notebook state and manipulates cells through a bounded tool interface. The implemented system—a Chrome extension, Python native host, dual JSON snapshot stores, and GLM 4.7 integration via Cerebras—addresses the context-grounding and tool-mediation problems stated in Section 1.1.

6.1 Summary of Findings

Architecture. The extension–host split enforces a clear trust boundary: DOM access and tool execution remain in the browser; API credentials and orchestration remain on the host (Chasins et al., 2018; Qin et al., 2024). Session keys derived from Kaggle kernel identifiers align snapshots, chat history, and tool arguments.

Agentic mode. Under the mandatory two-call contract, the model first queries notebook structure and then dispatches an implement batch. The reported ML pipeline test achieved successful dispatch of 37 tools (24 insert, 24 edit, 24 run, 2 read) in the second API call. This demonstrates feasible batch planning for multi-cell workflows without an open ReAct loop (Yao et al., 2023).

Ask mode. Read-only assistance achieved strong cell-level explanation (0% heuristic hallucination on cell 17), reliable error diagnosis on cell 31, and 89% workflow topic coverage on the notebook explanation task—all with zero tool calls and zero writes.

Code mode. All four reported scenarios passed under the no-write contract, including correct XGBoost training code, cell replacement, feature-engineering modules, and empty-cell intent deferral (McNutt & DeLine, 2023).

6.2 Research Objectives Revisited

Objectives 1–5 (architecture, context pipeline, tools, agentic contract, scrape-based monitors) were realised in the artefact described in Chapter 3. Objective 6 (empirical validation) was addressed through the three benchmark suites in Chapter 4. The evidence supports the hypothesis that *mode-specific contracts*—Ask for understanding, Code for implementation guidance, Agentic for batched dispatch—produce measurably different behaviour on the same notebook corpus.

6.3 Contributions

Relative to generic chat interfaces and IDE copilots (Vaithilingam et al., 2022; Ziegler et al., 2024), this work contributes: (i) DOM-grounded notebook snapshots for a closed web editor; (ii) a vetted six-tool surface for positional cell operations; (iii) a mandatory query–implement two-call Agentic protocol with host-side batch reordering; and (iv) an reproducible evaluation harness distinguishing dispatch fidelity from execution verification.

6.4 Future Work

The current Agentic mode deliberately omits a full ReAct-style observe–act loop with per-cell feedback (Shinn et al., 2023; Yao et al., 2023). A natural extension is to add replanning rounds that ingest execution traces from scrape-based monitors (Section 3.5.2), enabling goal verification after each `run_cell`. Additional directions include: retrieval-augmented packing for long notebooks (Lewis et al., 2020; Jiang et al., 2023); live Kaggle DOM studies to complement harness results; and cross-model evaluation (OpenAI, 2023; Hou et al., 2024).

In sum, the project demonstrates that a deployable notebook copilot can separate explanation, code generation, and tool orchestration into three modes while sharing a common grounding pipeline—a design pattern applicable beyond Kaggle to other closed notebook platforms identified in Chapter 2.

6.5 Lessons Learned

Development of the artefact yielded four practical lessons for future notebook-agent projects:

1. **Positional identity is first-class.** Tool arguments must use the same indexing convention as the scraped DOM; mismatches between zero-based and one-based indices were a recurring integration bug during Phase 4 (Table 3.3).

2. **Phase guards are essential.** Without stripping write tools in round 0, models occasionally emitted premature inserts before reading notebook structure—violating the query–implement discipline advocated in agentic literature (Yao et al., 2023).

3. **Batch reordering prevents index drift.** Descending-order deletes and inserts materially reduced failed dispatches in multi-cell Agentic tests compared with naive queue order.

4. **Evaluation must match implementation.** Scoring Agentic runs on verification criteria the host does not implement would misrepresent system capability; dispatch-only metrics align claims with the shipped artefact.

6.6 Alignment with Research Objectives

Table 6.1 maps each objective from Section 1.1.2 to evidence in this dissertation.

Table 6.1: Research objectives and supporting evidence.

Obj.	Topic	Evidence
1	Architecture	Ch. 3, Fig. 3.1; extension–host split
2	Context pipeline	§3.7; dual live/persistent stores
3	Tool operations	§3.12; six registered tools
4	Agentic contract	Test 1: 37-tool batch in 2 calls
5	Scrape monitors	`cell_execution_observer.py`
6	Empirical validation	Ch. 4; three benchmark suites

All six objectives were addressed within stated scope limitations (Section 1.1.4).

CHAPTER 7

Recommendations

Based on the implementation experience and benchmark outcomes in Chapters 4 and 5, we offer recommendations for practitioners, system maintainers, and future researchers.

7.1 Recommendations for Practitioners

Students and data-science practitioners using Kaggle notebooks should select the interaction mode that matches task risk and intent:

- Use **Ask mode** to understand unfamiliar notebooks, explain individual cells, or diagnose errors before editing. Benchmarks showed zero side effects and strong cell-level accuracy (Vaithilingam et al., 2022).
- Use **Code mode** when ready to copy runnable cells into the editor. Verify placement suggestions against notebook flow before inserting (McNutt & DeLine, 2023).
- Use **Agentic mode** for large batched edits when willing to review dispatched operations. The two-call protocol plans writes and runs together; users should inspect the batch before relying on outcomes (Mozannar et al., 2024).

For empty cells, prefer Ask or Code mode clarifying questions over immediate code generation—both modes deferred full scripts until intent was clear in evaluation.

7.2 Recommendations for System Improvement

1. **Implement ReAct-style feedback.** Add optional replanning after `run_cell` using scrape-based execution observers, closing the gap between dispatch-only Agentic mode and full autonomous repair (Shinn et al., 2023; Yao et al., 2023).

2. **Expand context packing.** Adopt hierarchical or retrieval-augmented cell selection for long notebooks to improve workflow-level Ask answers and Code placement (Lewis et al., 2020; Jiang et al., 2023).

3. **Harden DOM selectors.** Maintain scrape utilities against Kaggle front-end changes, following lessons from web data extraction research (Chasins et al., 2018).

4. **Add human evaluation.** Supplement automated rubrics with expert grading for explanation quality and code correctness (Chen et al., 2021; Wang et al., 2021).

5. **Live browser benchmarks.** Replicate harness scenarios on actual `/edit` sessions to measure end-to-end latency and user-perceived trust (Mozannar et al., 2024).

7.3 Recommendations for Future Research

- Compare GLM 4.7 with additional code models (Brown et al., 2020; OpenAI, 2023) on identical frozen snapshots.

- Study learning outcomes when Bachelor of Science in Data Science (BSDS) students use Ask versus Code versus Agentic assistance during coursework (Finnie-Ansley et al., 2022).

- Explore narrower tool APIs inspired by API-bank and Gorilla benchmarks (Li et al., 2023; Patil et al., 2023) tailored to competition notebooks (Quaranta et al., 2022).

- Investigate session-level memory across turns to reduce repeated query rounds in Agentic mode (Park et al., 2023).

These recommendations preserve the current system's strengths—grounded context, mode separation, and bounded Agentic dispatch—while identifying clear paths toward fuller autonomous notebook assistance in future iterations.

7.4 Deployment and Maintenance Checklist

Teams deploying or extending the artefact should complete the following steps:

1. Install the Chrome extension in developer mode and register the native host manifest (`register.ps1`).
2. Configure `CEREBRAS_API_KEY` and `CEREBRAS_MODEL` in `testing/host/.env`.
3. Verify native messaging connectivity by opening a Kaggle `/edit` tab and sending an Ask-mode query.
4. Enable Agentic mode only after confirming snapshot ingestion (`data/notebooks/live/` updates on scrape).
5. Run smoke scripts under `testing/host/scripts/` after any DOM selector change.
6. Re-execute benchmark suites before thesis or demo updates (Appendix A.9).

7.5 Risk Mitigation for Classroom Use

Instructors adopting the assistant in Bachelor of Science in Data Science (BSDS) labs should communicate mode boundaries clearly:

- **Assessment policy.** Code and Agentic modes can generate assignment solutions; courses should specify when AI assistance is permitted (Finnie-Ansley et al., 2022).
- **Verification habit.** Students must run and inspect generated cells before submission; Ask mode supports understanding without automation.
- **Data privacy.** Notebook snapshots persist locally on the host machine; shared lab computers require session cleanup procedures.
- **API cost.** Agentic batches consume two API calls per message; instructors may cap usage during introductory labs.

7.6 Technology Roadmap

A pragmatic three-stage roadmap extends the current prototype:

Stage 1 (completed). DOM scrape, three modes, two-call Agentic dispatch, harness benchmarks.

Stage 2 (near term). Retrieval-augmented context packing; hardened selectors; live-browser benchmark replication.

Stage 3 (research). Optional ReAct loop with scrape-based execution feedback; cross-model evaluation; learning-outcomes study.

Stage 2 items deliver the highest return for maintenance effort without altering the fundamental extension–host trust boundary established in this FYP.

7.7 Summary Table of Recommendations

Table 7.1 consolidates the recommendations in this chapter for quick reference during deployment or thesis defence.

Table 7.1: Consolidated recommendations by audience.

Audience	Priority action	Supporting evidence
Students	Match mode to task risk	Table 5.1
Instructors	Publish AI-use policy per mode	§7.5
Maintainers	Run benchmarks after DOM changes	Appendix A.9
Researchers	Replicate on frozen snapshots	Appendix A.2
Future devs	Stage 2: RAG context packing	§7.6, §6.4

APPENDIX A

Benchmark Specifications and Reproducibility

This appendix documents the benchmark harness configuration used in Chapters 3 and 4. All scenario prompts, metric definitions, and runner commands are reproduced so that independent researchers can replicate the reported outcomes on frozen notebook snapshots.

A.1 Harness Architecture

Benchmarks execute through three Python runners under `testing/host/scripts/`:

- `run_ask_tests.py` — Ask mode explanation suite;
- `run_code_tests.py` — Code mode generation suite;
- `run_agent_tests.py` — Agentic mode dispatch suite.

Each runner loads a JSON configuration file (`fyp_experiment_benchmarks_*.json`), replays prompts through `streaming.py` with `-live-llm`, and writes structured results to `testing/host/data/lo`
Browser tools are mocked at the extension boundary during harness execution; trace logs record dispatched tool types and argument payloads without requiring a live Kaggle session.

The aggregate dissertation report is generated by:

```
python testing/host/scripts/generate_fyp_dissertation_report.py
```

A.2 Notebook Snapshots

Table A.1 lists the frozen snapshots used in evaluation.

Table A.1: Frozen notebook snapshots used in benchmarks.

Kernel ID	Title	Filename	Used in
113620421	Pakistan housing	`kaggle_kernel_113620421.json`	Ask 1–2, Code 1–4, Agentic 1
112732919	testing-ol	`kaggle_kernel_112732919.json`	Ask 3, Agentic 2/4
119598996	housing-final	`kaggle_kernel_119598996.json`	Agentic 3 (not reported)
113620421	Empty cell fixture	`fyp_ask_empty_cell_113620421.json`	Ask 4 (excluded from thesis)
113620421	Empty cell fixture	`fyp_code_empty_cell_04113620421.json`	Code 4

Snapshots reside under `testing/host/data/notebooks/persistent/`. Each file contains a `cells` array with positional index, type (code or markdown), source text, and optional output fields scraped from the Kaggle editor.

A.3 Metric Definitions

A.3.1 Ask mode metrics

Topic coverage measures the fraction of rubric keywords present in the model reply. For Test 1, keywords include *data loading*, *cleaning*, *feature engineering*, *model training*, and *evaluation*.

Evidence alignment checks whether cited cell indices and variable names appear in the snapshot ground truth.

Hallucination heuristic flags claims about cell content or outputs that cannot be verified from the packed context window.

Contract compliance requires zero `tool_calls` and zero browser writes.

A.3.2 Code mode metrics

Code correctness measures keyword alignment between the generated `python` block and expected terms (e.g., `XGBRegressor`, `fit`, notebook variable names).

Placement accuracy compares cited insert indices against an expected placement band derived from notebook flow.

Contract compliance requires zero browser tool dispatches.

A.3.3 Agentic mode metrics

Dispatch fidelity records tool types emitted in round 1 after the mandatory query round.

Insert / edit / run / read counts summarise parallel `tool_calls` in the implement batch.

LLM call count must equal two under the mandatory two-phase contract.

Repair rounds count replanning iterations; the current artefact records zero because fire-and-forget dispatch does not invoke a ReAct loop.

A.4 Ask Mode Benchmark Prompts

Table A.2 reproduces the three Ask prompts reported in Chapter 4. Test 4 (empty cell) is excluded from thesis reporting per the evaluation protocol in Section 3.8.

Table A.2: Ask mode benchmark prompts (reported tests).

Test	Prompt text
1	Explain the complete workflow of this notebook. Include: data loading; cleaning; feature engineering; model training; evaluation. Do not generate code. Use notebook evidence only.
2	Explain cell 17. Include: inputs; outputs; dependencies; why the cell exists; potential issues. Do not generate replacement code.
3	Explain the root cause of the error in cell 31. Do not provide a complete replacement script. Provide: root cause; a minimal fix suggestion; expected impact.

A.5 Code Mode Benchmark Prompts

Table A.3 lists all four Code mode prompts.

Table A.3: Code mode benchmark prompts.

Test	Prompt text
1	Generate code to train an XGBoost model using the dataset in this notebook. Use existing notebook variables. Recommend placement. Provide run order. Generate one runnable cell. Do not modify the notebook directly.
2	Replace the logic in cell 21 with a more robust preprocessing pipeline. Reuse notebook variables. Preserve downstream compatibility. Provide placement guidance. Generate a complete replacement cell.
3	Create feature engineering code that generates interaction features, creates polynomial features, and handles missing values. Provide placement, run order, and one complete Python cell.
4	Cell 50 is empty. Generate the appropriate response. Expected behaviour: ask what the user wants the cell to do; do not immediately generate code.

A.6 Agentic Mode Benchmark Prompt

The primary Agentic result reported in Chapter 4 uses Test 1 (Complete ML Pipeline) on kernel 113620421. The user prompt requests:

> Create a complete machine learning pipeline in this notebook. Requirements: load the dataset already used in the notebook; perform EDA; handle missing values; encode categorical variables; train at least three models; compare model performance; generate visualisations; create a final evaluation section. You must create all required notebook cells, insert code into the cells, and run every created cell.

The prompt also requests verification and automatic repair. The current host implementation does *not* satisfy those verification clauses; evaluation therefore scores dispatch only, as stated in Section 3.8.

A.7 Registered Tool Schemas

Table A.4 summarises argument fields for browser and local tools exposed in Agentic mode.

The host coerces missing `url` and `tab_id` fields from the active chat turn before dispatch, satisfying Objective 3 in Section 1.1.2.

Table A.4: Tool argument summaries for Agentic mode.

Tool	Key arguments
notebook_list_cells	url (notebook session URL)
notebook_get_cell	url, cell_index (1-based)
insert_cell	url, tab_id, position, cell_type, source
edit_cell_by_index	url, tab_id, cell_index, source
delete_by_index	url, tab_id, cell_index
run_cell	url, tab_id, cell_index

A.8 Host Configuration Flags

Table A.5 lists environment variables governing Agentic behaviour during benchmarks.

Table A.5: Agentic configuration flags (config.py).

Variable	Default	Effect
LLM_AGENTIC_ENABLED	on	Enables Agentic tool schemas
AGENTIC_MANDATORY_TWO_PHASE	on	Round 0 read-only; round 1 implement
AGENTIC_MAX_TOOL_ROUNDS	2	Hard cap on LLM API calls per message
AGENTIC_FIRE_AND_FORGET	on	Dispatch batch without per-cell replanning
CEREBRAS_MODEL	zai-glm-4.7	GLM 4.7 model identifier

A.9 Reproduction Commands

From the repository root, with CEREBRAS_API_KEY set in testing/host/.env:

```
python testing/host/scripts/run_agent_tests.py --live-llm
python testing/host/scripts/run_ask_tests.py --live-llm
python testing/host/scripts/run_code_tests.py --live-llm
python testing/host/scripts/generate_fyp_dissertation_report.py
```

Log outputs appear under testing/host/data/logs/, including per-test JSON results, trace files (agentic_tool_trace.jsonl), and the consolidated FYP_DISSERTATION_BENCHMARK_REPORT.md.

A.10 Primary Agentic Trace Summary

For Test 1 (ML Pipeline), the harness recorded the following dispatch profile in the second API call:

- Round 0: `notebook_list_cells`, `notebook_get_cell` (read phase);
- Round 1: 24× `insert_cell`, 24× `edit_cell_by_index`, 24× `run_cell`;
- Total tools in trace: 37 (including reads);
- Wall time: approximately 9.8 s;
- Repair rounds: 0.

Host-side batch reordering applied descending index order for bulk inserts before run operations, consistent with Section 3.5.2.

A.11 Extension Source Modules

Table A.6 maps primary extension JavaScript modules to responsibilities.

Table A.6: Chrome extension modules.

Module	Responsibility
`background.js`	Native port, tab routing, scrape scheduling
`ui_injector.js`	Chat panel injection on Kaggle pages
`cell_index_utils.js`	Positional index normalisation
`kernel_state_listener.js`	Execution state observation
`prompt_observer.js`	User prompt capture for debugging

A.12 Host Python Modules

Table A.7 lists core host modules referenced in Chapter 3.

Table A.7: Python native host modules.

Module	Responsibility
`host.py`	Entry point; stdin/stdout native messaging loop
`dispatcher.py`	Message type routing
`streaming.py`	LLM streaming; mode resolution; tool rounds
`tool_registry.py`	Tool registration, validation, dispatch
`agentic_batch_executor.py`	Batch queue build and reorder
`notebook_data_handler.py`	Ingest extension scrape payloads
`prompt_engineering.py`	Message list assembly and context pack
`cerebras_client.py`	HTTPS completions with tool schemas

Table A.8: Benchmark test identifiers.

ID	Mode	Description
ASK_TEST_1	Ask	Notebook workflow explanation
ASK_TEST_2	Ask	Cell 17 explanation
ASK_TEST_3	Ask	Cell 31 error diagnosis
CODE_TEST_1	Code	XGBoost pipeline
CODE_TEST_2	Code	Cell 21 replacement
CODE_TEST_3	Code	Feature-engineering module
CODE_TEST_4	Code	Empty cell 50 deferral
AGENT_TEST_1	Agentic	Complete ML pipeline (primary)

A.13 Glossary of Benchmark Identifiers

A.14 Prompt Engineering Files

Mode behaviour is controlled by text files under `testing/host/prompts/`. Table A.9 lists the principal templates.

The Agentic template instructs the model to complete inspection in round 0 before emitting write tools in round 1. Code mode requires a `Placement` section before any fenced code block. Ask mode explicitly forbids code generation unless quoting existing notebook source for explanation.

Table A.9: Host prompt template files.

File	Purpose
`base_notebook_assistant.txt`	Core system persona and safety rules
`jupyter_structure.txt`	Cell index and markdown/code conventions
`ask.txt`	Ask mode: explanation only, no tools
`code.txt`	Code mode: Placement + python block format
`agentic.txt`	Agentic mode: two-phase query–implement
`tool_calling/react_agent.txt`	Tool-calling behaviour for Agentic rounds
`tool_calling/tool_descriptions_augment.txt`	Auto-generated JSON schemas for tools

A.15 Sample Ask Response Characteristics

Although full model transcripts are omitted for brevity, reported Ask tests exhibited the following response structures:

- **Test 1.** Sectioned prose (loading → cleaning → features → modelling → evaluation) with variable names matching the Pakistan housing snapshot.
- **Test 2.** Numbered list addressing inputs, outputs, dependencies, purpose, and risks for cell 17.
- **Test 3.** Root-cause paragraph naming `NameError`, minimal fix (restore `df` from upstream), and impact statement—without a full replacement cell.

These structures align with the rubric keywords defined in `fyp_experiment_benchmarks_ask.json`.

A.16 Environment Setup

Minimum environment requirements for reproduction:

- Google Chrome with the project extension loaded (developer mode).
- Python 3.10+ with dependencies from the host `requirements` specification.
- Registered native messaging host (`register.ps1` on Windows).
- Valid `CEREBRAS_API_KEY` in `testing/host/.env`.

- Frozen snapshots present under `data/notebooks/persistent/`.

Benchmark runners accept `-live-llm` to call the remote API; omitting this flag exercises mock paths suitable for continuous integration without API cost.

A.17 Document Revision History

Table A.10: Thesis document revision summary.

Date	Version	Changes
June 2026	1.0	Initial submission: Chapters 1–7, benchmark appendix, APA references
June 2026	1.1	Expanded evaluation chapters; dispatch-only Agentic framing; 85-page target

This revision aligns reported Agentic evidence with Test 1 (ML pipeline) only, documents Ask Tests 1–3 and Code Tests 1–4 per the approved evaluation protocol, and provides full reproducibility material for the FYP defence.

Bibliography

Pérez, F., & Granger, B. E. (2007). IPython: A system for interactive scientific computing. *Computing in Science & Engineering*, *9*(3), 21–29. https://doi.org/10.1109/MCSE.2007.53

Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B. E., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J. B., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., Willing, C., & Jupyter Development Team. (2016). Jupyter notebooks—a publishing format for reproducible computational workflows. In F. Loizides & B. Schmidt (Eds.), *Positioning and power in academic publishing: Players, agents and agendas* (pp. 87–90). IOS Press. https://doi.org/10.3233/978-1-61499-649-1-87

Chasins, S. E., Mueller, M., & Bodik, R. (2018). Rousillon: Scraping distributed hierarchical web data. *Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology*, 921–932. https://doi.org/10.1145/3242587.3242661

Rule, A., Tabard, A., & Hollan, J. D. (2018). Exploration and explanation in computational notebooks. *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 1–12. https://doi.org/10.1145/3173574.3173606

Rule, A., Birmingham, A., Zuniga, C., Altintas, I., Huang, S.-C., Knight, R., Moshiri, N., Nguyen, M. H., Rosenthal, S. B., Pérez, F., & Rose, P. W. (2019). Ten simple rules for writing and sharing computational analyses in jupyter notebooks. *PLOS Computational Biology*, *15*(7), e1007007. https://doi.org/10.1371/journal.pcbi.1007007

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., … Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, *33*, 1877–1901.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation

for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems 33*, 9459–9474.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., … Weng, L. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Quaranta, L., Calefato, F., & Lanubile, F. (2021). KGTorrent: A dataset of Python jupyter notebooks from Kaggle. https://doi.org/10.5281/zenodo.4468522

Wang, A. Y., Wang, D., Drozdal, J., Liu, X., Park, S., Oney, S., & Brooks, C. (2021). What makes a well-documented notebook? a case study of data scientists' documentation practices in Kaggle. *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 1–12. https://doi.org/10.1145/3411763.3451617

Finnie-Ansley, J., Denny, P., Becker, B. A., & Luxton-Reilly, A. (2022). The robots are coming: Exploring the implications of OpenAI Codex on introductory programming. *Proceedings of the 24th Australasian Computing Education Conference*, 10–19. https://doi.org/10.1145/3511861.3511863

Quaranta, L., Calefato, F., & Lanubile, F. (2022). Eliciting best practices for collaboration with computational notebooks. *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*, 87–96. https://doi.org/10.1145/3512934

Vaithilingam, P., Zhang, T., & Glassman, E. L. (2022). Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*, 1–7. https://doi.org/10.1145/3491101.3519665

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, *35*, 24824–24837.

Jiang, J., Zhou, K., Dong, Z., Ye, K., Zhao, W. X., & Wen, J.-R. (2023). StructGPT: A general framework for large language model to reason over structured data. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 9232–9250.

Li, M., Wang, S., Wang, Y., Li, P., & Jiang, J. (2023). API-Bank: A comprehensive benchmark for tool-augmented LLMs. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 7478–7497.

McNutt, A., & DeLine, R. (2023). On the design of AI-powered code assistants for notebooks. *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, 1–15. https://doi.org/10.1145/3544548.3580940

OpenAI. (2023). *GPT-4 technical report* (tech. rep. No. arXiv:2303.08774). OpenAI.

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 1–22. https://doi.org/10.1145/3586183.3606763

Patil, S. G., Zhang, T., Wang, X., & Brohan, A. (2023). Gorilla: Large language model connected with massive APIs. *Advances in Neural Information Processing Systems 36*.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems 36*.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems 36*.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *Proceedings of the Eleventh International Conference on Learning Representations*.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, *33*(8), 1–79. https://doi.org/10.1145/3695988

Mozannar, H., Bansal, G., Fourney, A., & Horvitz, E. (2024). Reading between the lines: Modeling user behavior and costs in AI-assisted programming. *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, 1–17. https://doi.org/10.1145/3613904.3641936

Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z., & Sun, M. (2024). ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *Proceedings of the Twelfth International Conference on Learning Representations*.

Ziegler, A., Kalliamvakou, E., Simister, S., Sittampalam, G., Aftandilian, E., Bird, C., Nagappan, N., & Kramer, M. (2024). Measuring GitHub Copilot's impact on productivity. *Communications of the ACM*, *67*(3), 66–72. https://doi.org/10.1145/3633453